

UNIT 3: Memory Management

3.1 Memory Management without Swapping or Paging:

- Monoprogramming without Swapping & Paging
- Multiprogramming and Memory Usage
- Multiprogramming and Fixed Partition

3.2 Swapping:

- Memory Management with Bit Maps
- Memory Management with Linked Lists
- Memory Management with Buddy System
- Allocation of Swap Space
- Analysis of Swapping Systems

3.3 Virtual Memory:

- Paging
- Page Tables

- Example of Paging Hardware
- Associative Memory

3.4 Page Replacement Algorithms:

- The optimal Page Replacement Algorithms
- The First-in, First-out
- The Second Chance Page Replacement Algorithms
- The Least Recently Used
- Modeling Paging Algorithms (Stack Algorithm)

3.5 Segmentation:

- Implementation of Pure Segmentation
- Segmentation with Paging: MULTIC
- Segmentation with Paging: The Intel

Memory Management

- Main memory (**RAM**) is an important resource that must be carefully managed.
- The entire program and data of a process must be in main memory for the process to execute.
- The OS component that is responsible for managing memory related to the following motivations is called **memory manager**:
 - How to keep the track of processes currently being executed?
 - Which processes to load when memory space is available?
 - How to load the processes that are larger than main memory?
 - How do processes share the main memory?

Memory Management Issues

- Uniprogramming or Multiprogramming system
- Absolute or relocatable partitions
- Multiple fixed or multiple variable partitions.
- Partition in rigid manner or dynamic.
- Program run from specific partition or anywhere
- Job placing at contiguous block or any available slot

TU Exam Question 2067/TU Model Question

- What are the main motivations and issues in primary memory management?

Memory Management Requirements

Relocation:

- Programmer does not know where the program will be placed in memory when it is executed.
- While the program is executing, it may be swapped to disk and returned to main memory at a different location (relocated).
- Memory addresses must be translated in the code to actual physical memory addresses.
- Hardware used - Base (relocatable) register, limit register.

Memory Management Requirements...

Protection:

- Prevent processes from interfering with the OS or other processes.
- Processes should not be able to reference memory locations in another process without permission.
- Impossible to check absolute addresses in programs since the program could be relocated.
- Often integrated with relocation.

Memory Management Requirements...

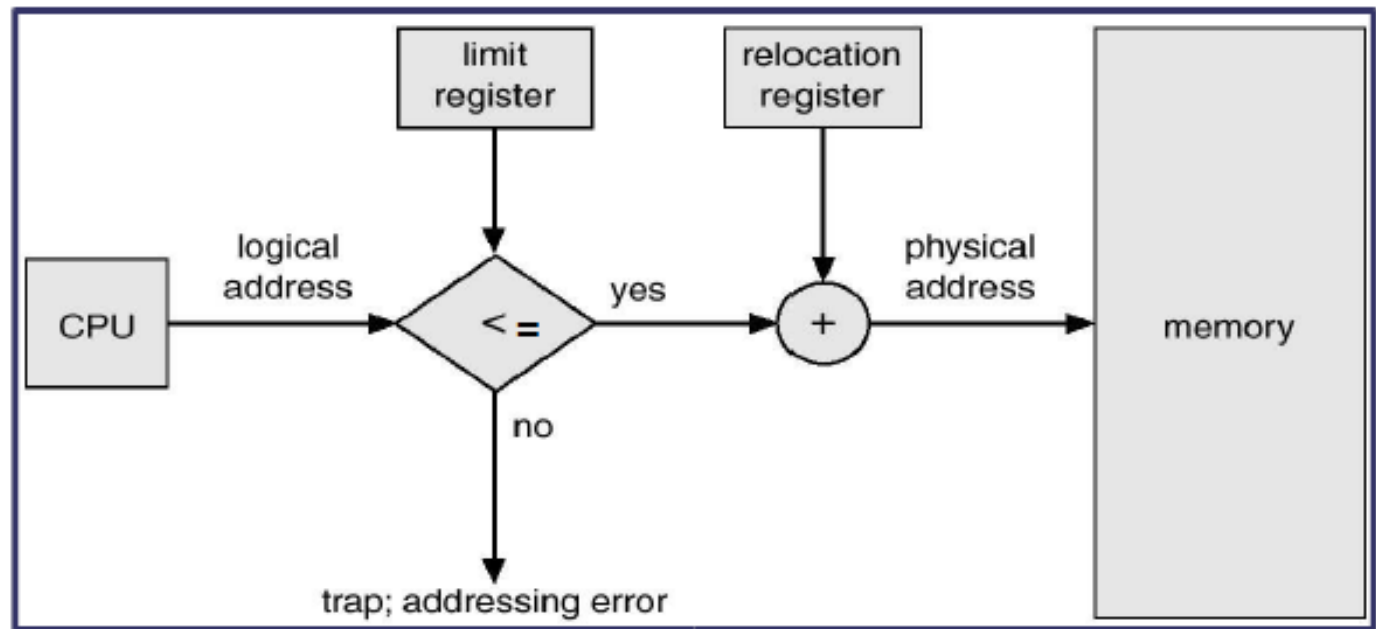
Sharing:

- Allow several processes to access the same portion of memory (allow to share data).
- Better to allow each process access to the same copy of the program rather than have their own separate copy.

Logical- Versus Physical-Address

- **Logical Address**: The address generated by CPU. It is also known as ***virtual address***.
- **Physical Address**: Actual address seen by the memory unit i.e., loaded into the ***memory-address register***.
- Mapping from logical address to physical address is called ***relocation*** and it is done by a hardware device called the memory-management unit (**MMU**).
- Two registers: **Base** and **Limit** are used in mapping.
- This mechanism is also used in memory protection - memory outside the range is protected.

Mapping from logical to physical address



Base register (relocation): Holds smallest legal physical memory address.

Limit register: contain the size of range.

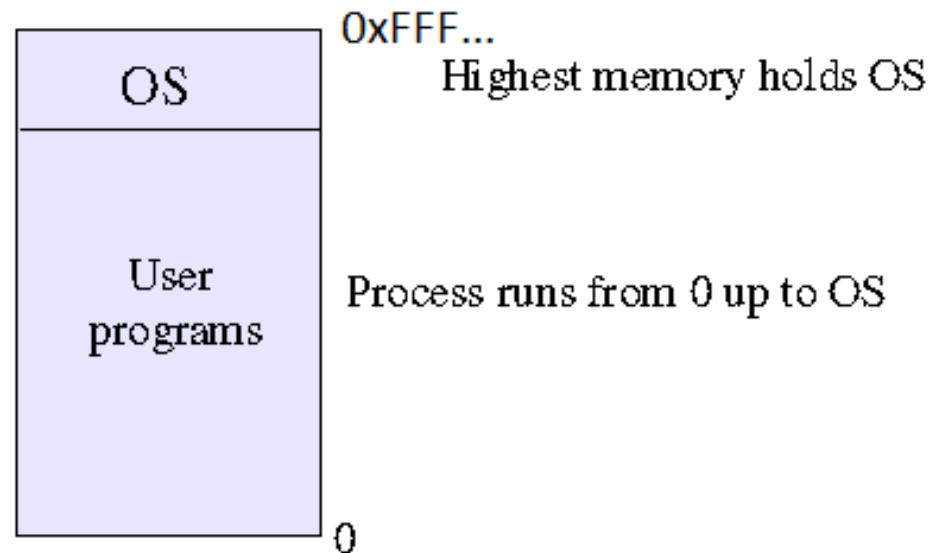
Example:

logical address = 300

if Base = 1500

physical address = $1500 + 300 = 1800$.

Uniprogramming Model



Run one program at a time with single segment per process. Memory space is divided into two partitions by convention; one partition is allocated to OS and the other to executing process.

Uniprogramming Model...

- As soon as the user types the command, the OS copies the requested program from disk to memory and execute it.
- Used in early Dos system.
- Also called **Monoprogramming**.
- **Disadvantages:**
 - Only one process can run at a time.
 - Processes can destroy OS (an erroneous process can crash the entire system).

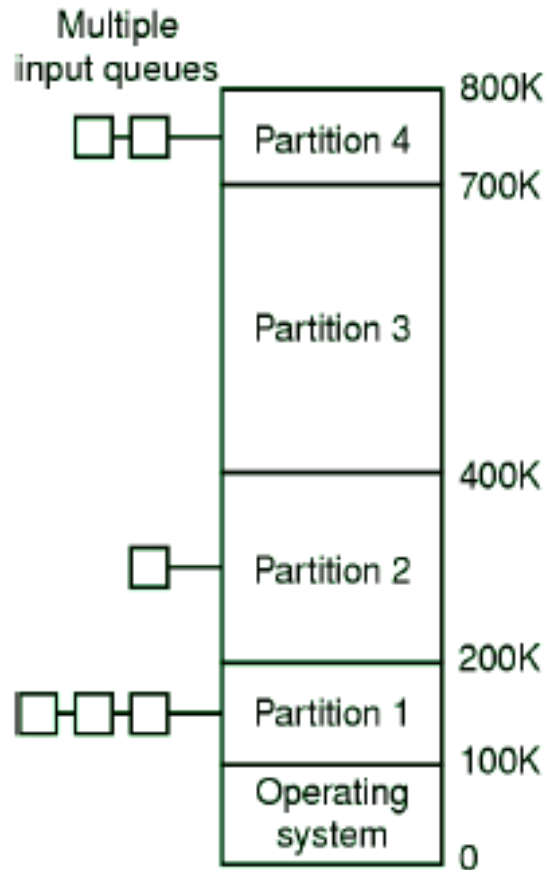
Multiprogramming Model

- In multiprogramming, several processes are in memory at the same time.
- To allow multiprogramming model, two approaches exist:
 - *Fixed-Partition Multiprogramming*
 - *Variable-partition Multiprogramming*
- If size of processes is larger than memory, then other approaches are used:
 - *Swapping*
 - *Virtual Memory*

Fixed-Partition Multiprogramming

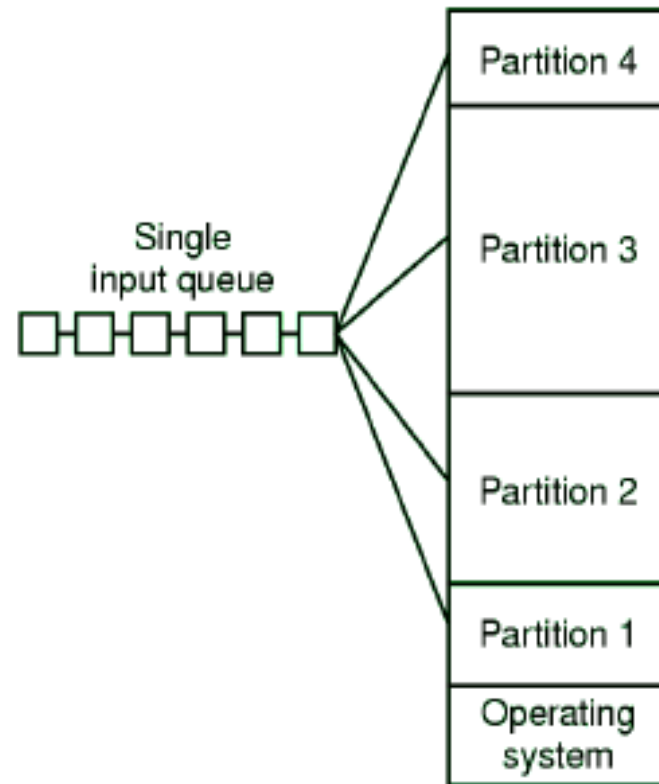
- In this method, multiple partitions are created to allow multiple user processes to reside in memory simultaneously.
- The partitions are fixed (possibly unequal), can be done using base and limit register.
- The degree of multiprogramming is bound by the number of partitions.
- Partition table stores the partition information for a process.
- When a job arrives, it can be put into the **input queue** (The collection of processes on the disk that is waiting to be brought into memory for execution is called the **input queue**.) for the smallest partition large enough to hold it. Since the partitions are fixed, any space in partition not used by a process is lost.
- Used by OS/360 on large mainframes.
- Implemented in two ways :
 - Dedicate partitions for each process (Absolute translation).
 - Maintaining a single queue (Relocatable translation).

Fixed Partition Multiprogramming



(a)

Absolute Translation



(b)

Relocatable Translation

Dedicating partitions

- Separate input queue is maintained for each partition.
- Processes run only in dedicated partition.
- **Problem:** If a process is ready to run and its partition is occupied then that process has to wait even if other partitions are available ... wastage of storage.

Maintaining a Single Queue

- Maintains a single queue for all processes.
- When a partition becomes free, the process closest to the front of queue that fits in it could be loaded into the empty partition and run.
- To not waste the large partition for small process, another strategy is used i.e. to search the whole input queue and whenever a partition becomes free, pick the largest process that fits in the partition.
- **Problem:**
Wastage of memory when many processes are small.

Variable Partition Multiprogramming

- How to allocate the memory for a process exactly as much as they need? - ***variable partition multiprogramming***.
- In variable partition multiprogramming, when processes arrive, they are given exactly as much storage as they need.
- When processes finish, they leave holes in main memory; OS fills these holes with another process from the input queue of processes.
- **Advantage:**
Processes get the exact size partition of their request for memory ... No waste of memory.
- **Problem:**
When holes are given to other processes, the holes may again get partitioned, the remaining holes getting smaller and smaller and eventually becoming too small to hold new processes so that waste of memory occurs.

Variable Partition Multiprogramming...

Input queue

PA 25K	PB 15K	PC 20 K	PD 20 K	
--------	--------	---------	---------	--

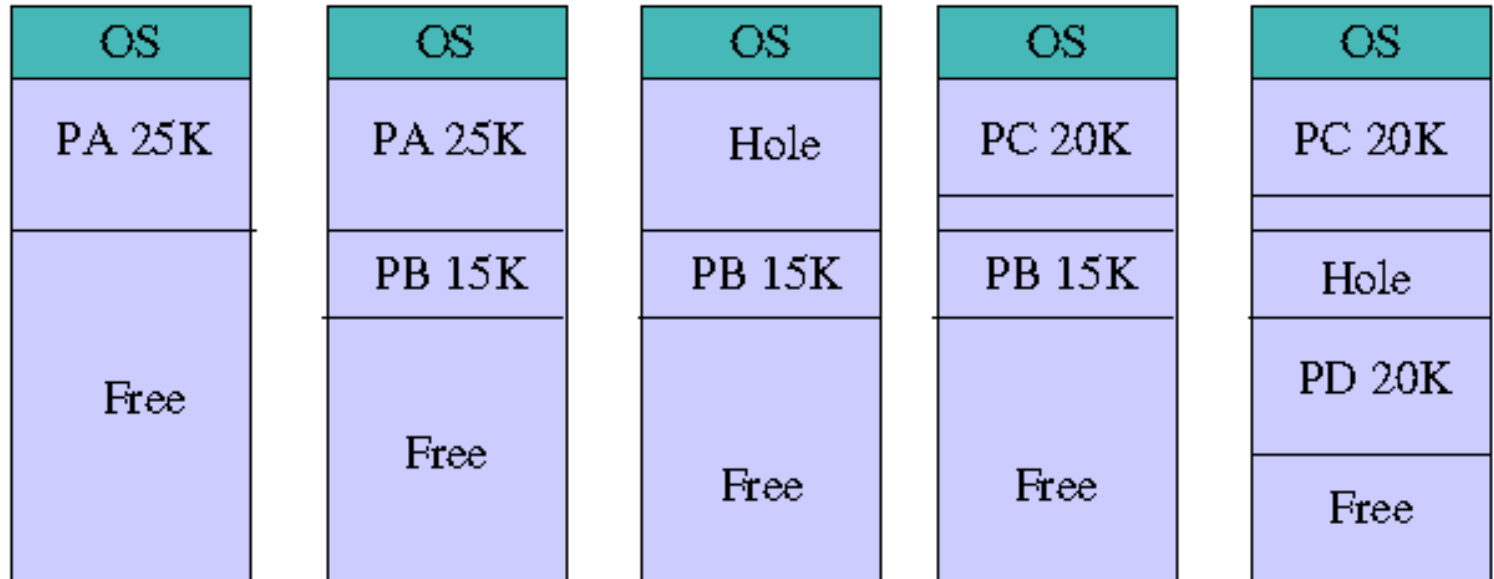


Fig: Memory allocation and holes

Variable Partition Multiprogramming...

- **Solution: Memory Compaction**

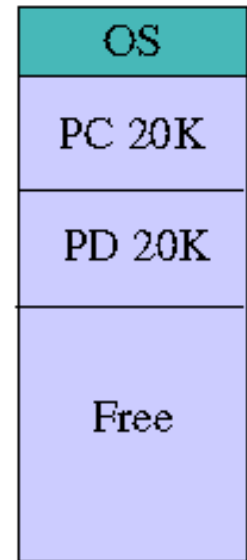
By moving processes in memory, the memory holes can be collected into a single section of unallocated space – memory compaction.

Problem:

It is highly expensive – it requires a lot of CPU time;

Eg: On a 256-MB machine that can copy 4 bytes in 40 nsec, it takes about 2.7 sec to compact all memory.

It stops every thing when compaction.



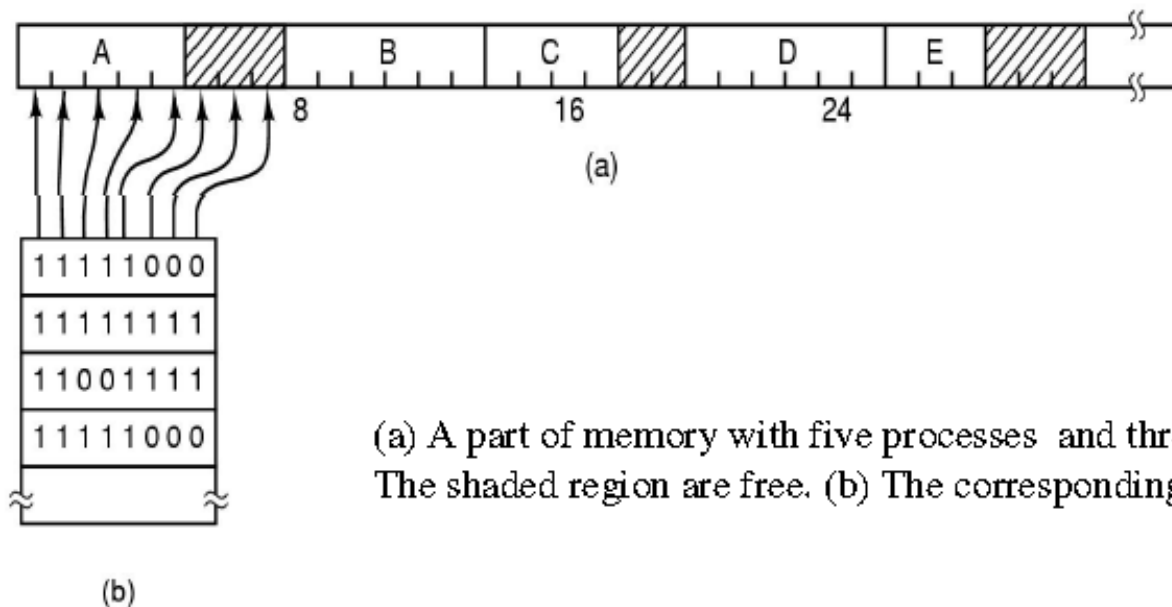
Memory compaction

Variable Partition Multiprogramming...

Memory Management with Bitmaps

Bitmap Management

Memory is divided up into allocations units and each bit in the map indicates whether or not the corresponding unit is allocated.



(a) A part of memory with five processes and three holes. The shaded regions are free. (b) The corresponding bitmap.

Bitmap Management...

- Each allocation unit is a bit in the bit map, which is 0 if the unit is free, and 1 if it is occupied (or vice versa).
- When a process comes into the memory, it searches for required number of consecutive 0 bits in the map.
- The size of the allocation unit is an important design issue. The smaller the allocation unit, the larger the bitmap. Increasing the size of allocation unit decreases the size of bitmap, but increases the amount of memory wasted in the last unit of the process when the process size is not a multiple of the allocation unit.
- **Problem:** To bring a process of k allocation units into memory, the memory manager must search the bitmap to find a run of k consecutive 0 bits in the map. Searching a bitmap for a run of a given length is a slow operation.

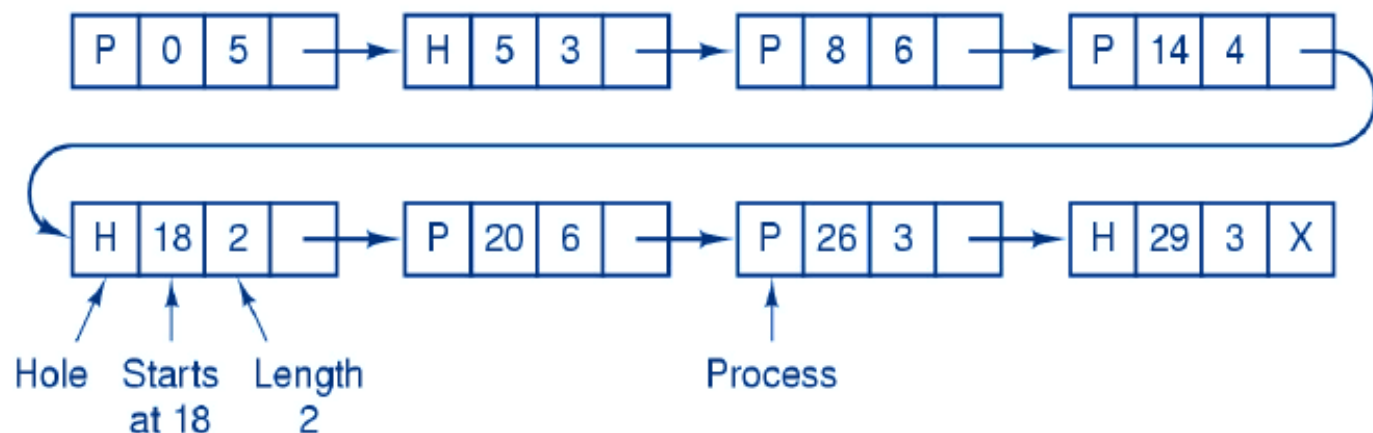
Variable Partition Multiprogramming...

Memory Management with Linked Lists

Linked List Management

Maintains a linked list of allocated and free memory segments, where a segment is either a process or a hole between the two process.

Each segment contains an entry indicating the segment size and a pointer to the next segment in the list.



Linked List Management...

- H represents hole and P represents process and the segment list is kept sorted by address. Sorting has an advantage that when a process terminates, updating the list is straightforward.
- It may be implemented as doubly-linked list to make a more convenient search.
- When a process terminates, if any neighbor is already a hole, the holes are merged called **coalescing**.

Before B terminates



After B terminates



Problem : Still some form of fragmentation

Partition Selection Algorithms

- **Situation:** Multiple memory holes are large enough to contain a process, OS must select which hole the process will be loaded into.
- Three approaches (algorithms):
 - **First Fit**
 - **Best Fit**
 - **Worst Fit**

First Fit

- Allocate the first hole that is big enough. It stops the searching as soon as it find a free hole that is large enough. The hole is then broken up into two pieces, one for the process and one for unused memory.
- **Advantage:** It is a fast algorithm because it searches as little as possible.
- **Problem:** Not good in terms of storage utilization.

Best Fit

- Allocate the smallest hole that is big enough. It searches the entire list, from beginning to end, and takes the smallest hole that is adequate. Rather than breaking up a big hole that might be needed later, it finds a hole that is close to the actual size needed.
- **Advantage:** More storage utilization than first fit.
- **Disadvantage:** Slower than first fit because it requires to search whole list every time. Also it creates tiny holes that may be useless in the end and eventually may result in more wastage of memory than even first fit.

Worst Fit

- Allocate the largest available hole. It searches the entire list, and takes the largest hole. Rather than creating tiny holes, it produces the largest leftover hole, which may be more useful (as other processes may fit there).
- **Advantage:** Sometimes it has more storage utilization than first fit and best fit.
- **Disadvantage:** Slower than first fit because it requires to search whole list every time.

Reading: Pages 185-187 (Tanenbaum)

Homework

Given the memory partitions of 100KB, 500KB, 200KB, 300KB, & 600KB (in order), how would each of the first fit, best fit and worst fit algorithms place processes of 212KB, 417KB, 112KB, & 426KB (in order)? Which algorithm makes the most efficient use of memory?

Answer:

First Fit: 212K is put in 500K partition, 417K is put in 600K partition, 112K is put in 288K partition (new partition $288K = 500K - 212K$), and 426K must wait.

Best Fit: 212K is put in 300K partition, 417K is put in 500K partition, 112K is put in 200K partition and 426K is put in 600K partition.

Worst Fit: 212K is put in 600K partition, 417K is put in 500K partition, 112K is put in 388K partition and 426K must wait.

In this example, Best-fit turns out to be the best.

Swapping

- **Until now:** User processes remain in main memory until completion and have contiguous allocation.
- In timesharing system, the situation may be different i.e. sometimes there is not enough memory to hold all processes.
- How to handle this problem? – **swapping** i.e. excess processes that cannot be loaded into memory must be kept on the disk and when there is enough room for them to run, they are brought into memory and run dynamically.
- A process, however, can be swapped temporarily out of the memory to disk, and then brought back into memory for continued execution.
- In timesharing system, processes will be swapped in and out many times before its completion.

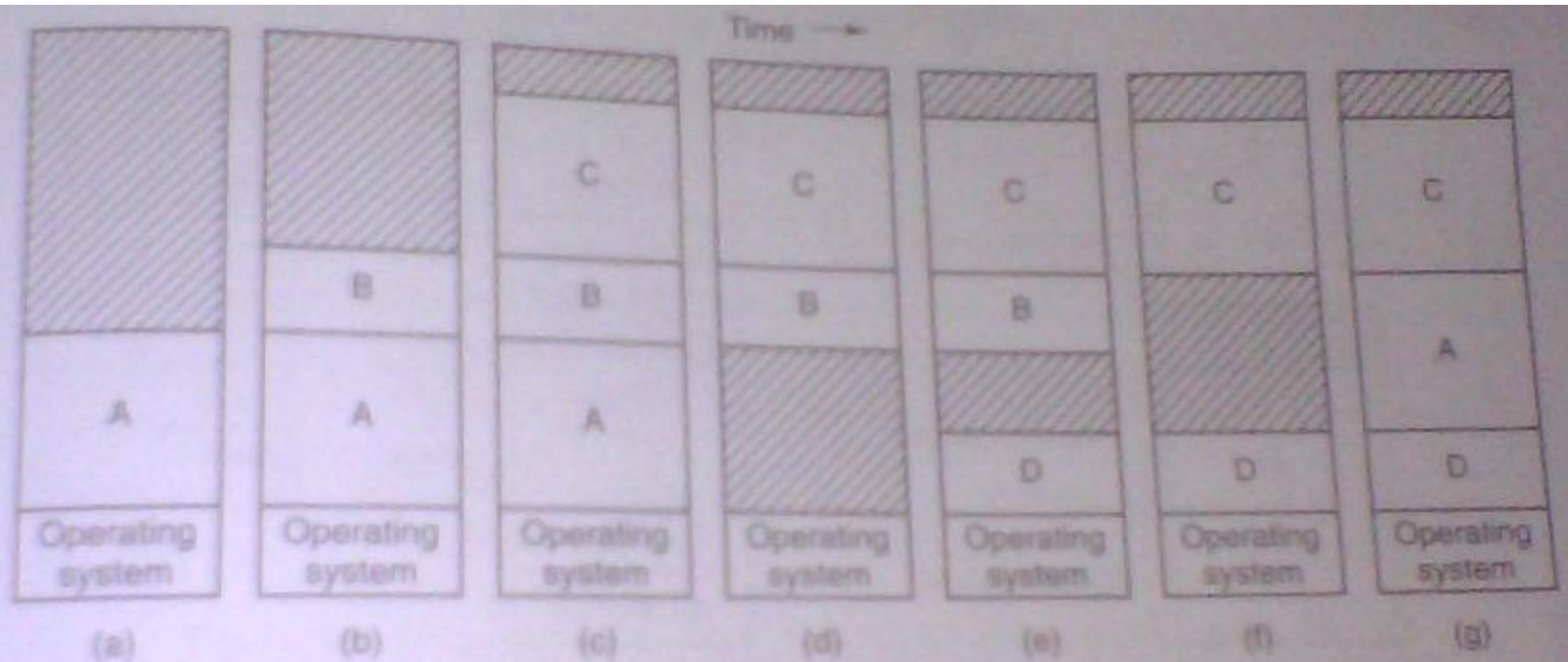


Figure Memory allocation changes as processes come into memory and leave it. The shaded regions are unused memory.

Overlays

- **What to do when the program's size is bigger than the available memory?**
- In the 1960s, when the programs were too big to fit in available main memory, the solution adopted was to split the program into little pieces called overlays.
- Overlay 0 would start running first; when it was done, it would call overlay 1 next and so on.
- The overlays were kept on disk and swapped in and out by OS.
- **Problem:** The job of splitting program into pieces (making overlays) had to be done by programmer; time consuming, boring and error prone for programmer.
- **Solution: VIRTUAL MEMORY**

Virtual Memory

- Virtual memory is a concept that is associated with ability to address a memory space much larger than that the available physical memory.
- The basic idea behind the virtual memory is that the combined size of the of the program, data, and stack may exceed the amount of physical memory available for it. The OS keeps those part of the program currently in use in main memory, and the rest on the disk.
- Virtual memory can be implemented by two most commonly used methods : **Paging** and **Segmentation** or mix of both.

Virtual address space vs. Physical address space

- The set of all virtual (logical) addresses generated by a program is called a virtual address space; the set of all physical addresses corresponding to these virtual addresses is a physical address space.
- **MMU**: The run time mapping from virtual address to physical address is done by a special hardware device called memory management-unit (MMU).

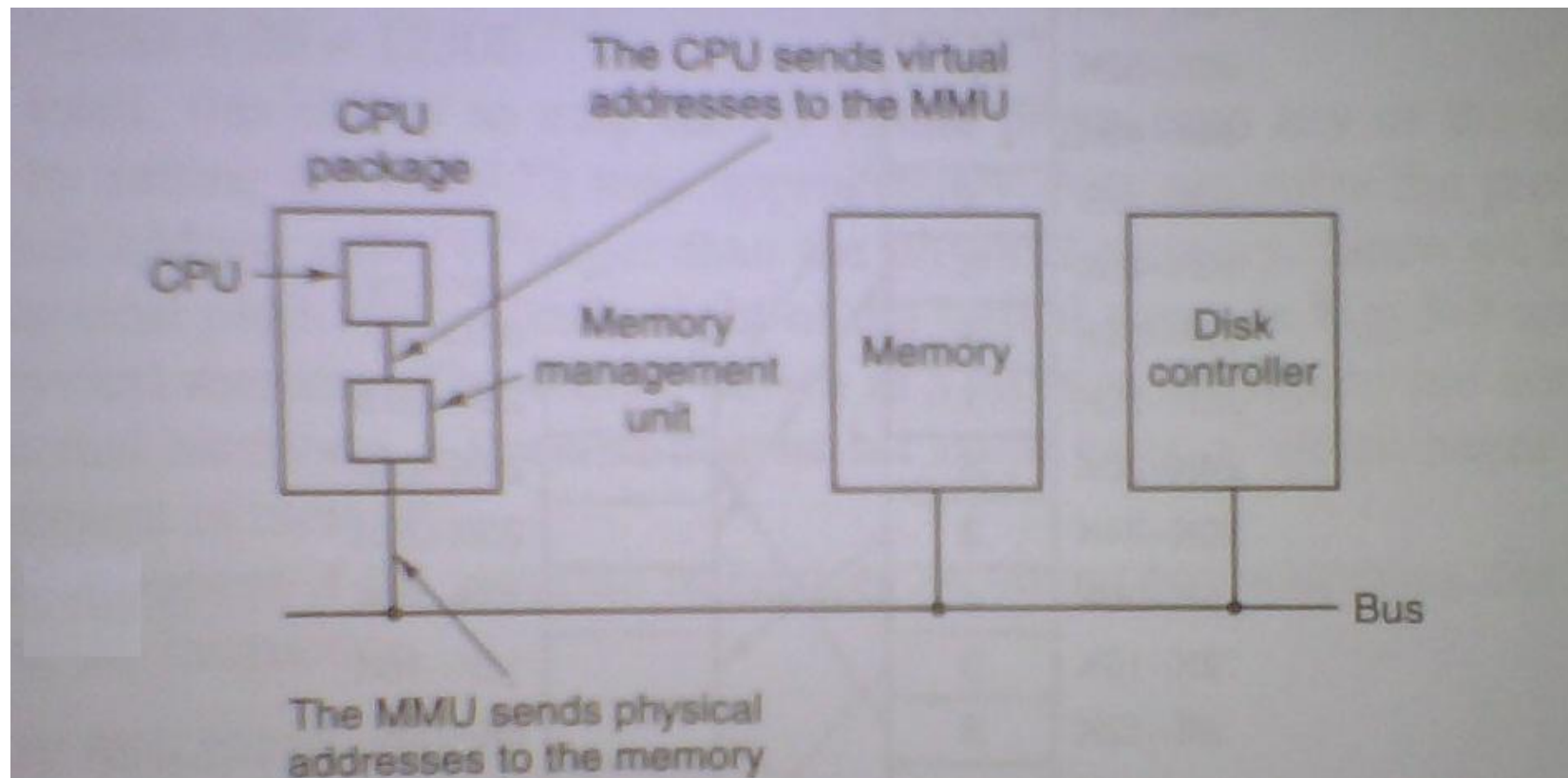
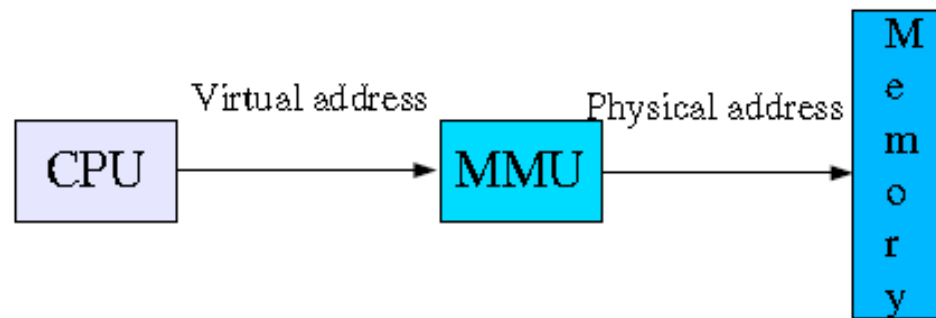
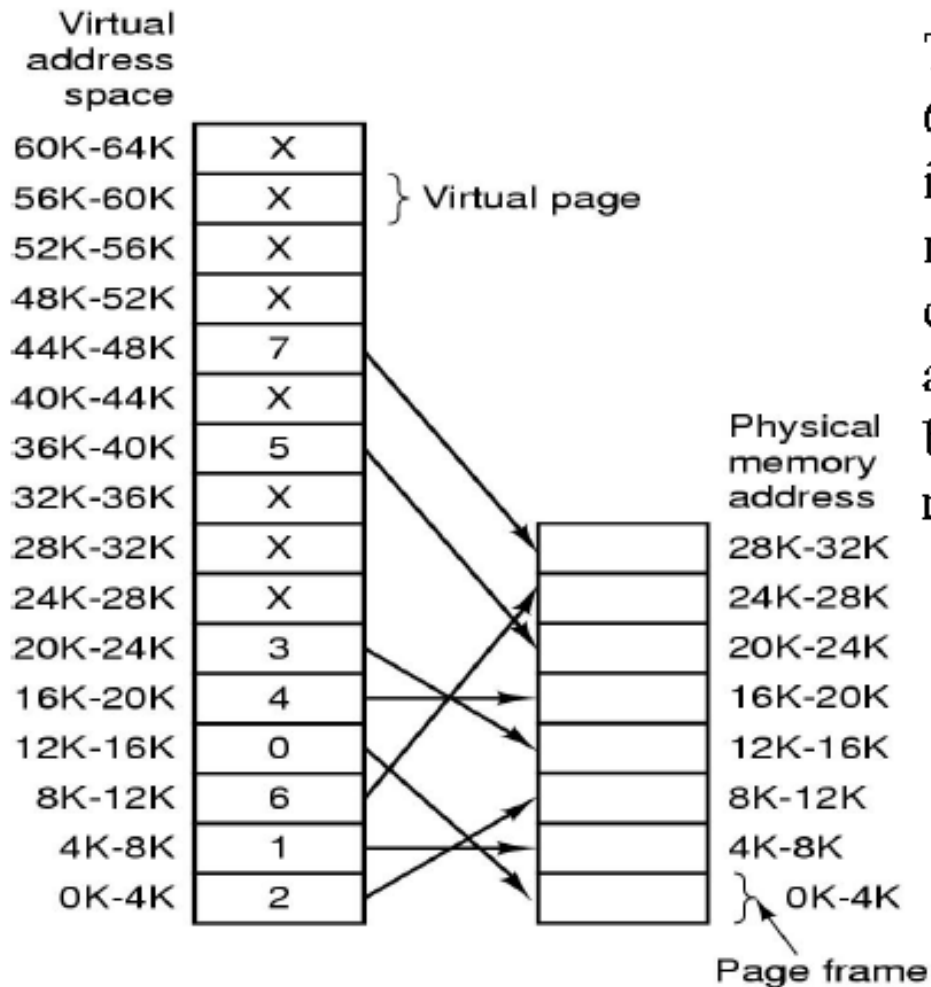


Figure : The position and function of the MMU. Here the MMU is shown as being a part of the CPU chip because it commonly is nowadays. However, logically it could be a separate chip and was in years gone by.

Paging

- The virtual address space (process) is divided into fixed sized blocks called **pages** and the main memory is also divided into blocks of the same size called **frames**.
- When a process is to be executed, its pages are loaded into any available memory frames from the backing store.
- The size of the pages is determined by the hardware, normally from 512 bytes to 64KB (in power of 2).
- Paging permits the physical address space of a process to be noncontiguous.
- Traditionally, support for paging has been handled by hardware, but the recent design have implemented by closely integrating the hardware and OS.

Paging



This example shows 64KB program can run in 32KB physical memory. The complete copy is stored in the disk and the pieces can be brought into memory as needed.

Paging...

- With 64KB of virtual address space and 32KB of physical memory and 4KB page size, we get 16 virtual pages and 8 frames.
- **Note:** The range marked 0K-4K means that the virtual or physical addresses in that page are 0 to 4095. The range 4K-8K refers to addresses 4096 to 8191, and so on. Each page contains exactly 4096 addresses starting at a multiple of 4096 and ending one shy of a multiple of 4096.

Paging...

- What happens in following instruction?

`MOV REG, 0`

- This virtual address, 0, is sent to the MMU. The MMU sees that this virtual address falls in page 0 (0-4095), which is mapping to page frame 2 (8192 -12287). Thus the address 0 is transformed to 8192 and the output address 8192 is put onto the bus. The memory knows nothing at all about the MMU and just sees a request for reading or writing address 8192, which it honors. Thus, the MMU has effectively mapped all virtual addresses between 0 and 4095 onto physical addresses 8192 to 12287.
- Similarly, the instruction
`MOV REG, 8192`
is effectively transformed into
`MOV REG, 24576`

Paging...

- Also, the virtual address 20500 is mapped to physical address 12308. (**HOW???**)
- What happens if the program references an unmapped address, for example, by using the instruction

MOV REG, 32780

which is byte 12 within virtual page starting at 32768? The MMU notices that the page is unmapped (indicated by a cross in the figure) and causes the CPU to **trap** to the operating system. This trap is called a **page fault**. The operating system picks a little-used frame and writes its contents back to the disk. It then fetches the page just referenced into the page frame just freed, changes the map, and restarts the trapped instruction.

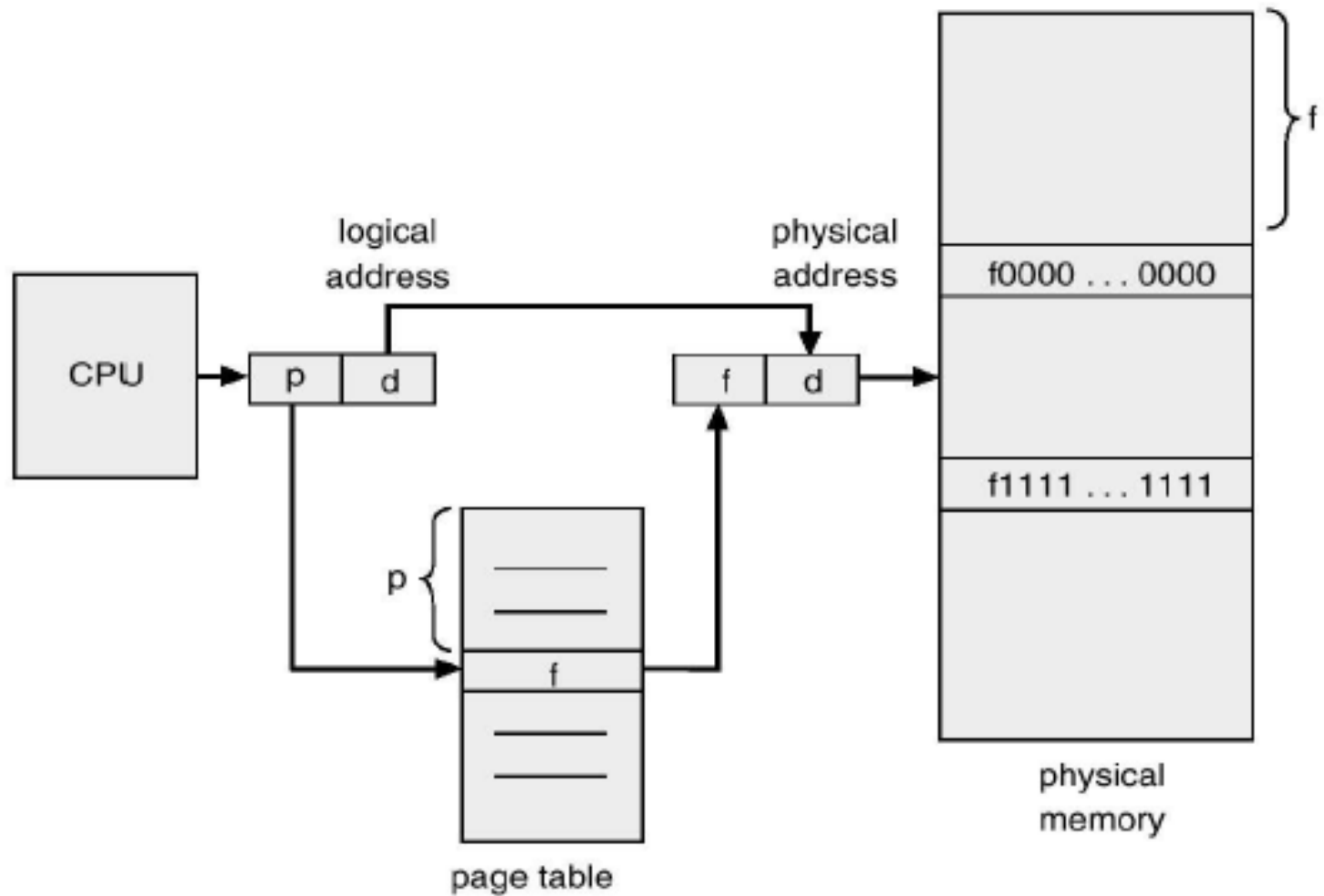
Paging...

- For example, if the OS decided to evict page frame 1, it would load virtual page 8 at page frame 1 and make two changes to the MMU map. First, it would mark virtual page 1's entry as unmapped, to trap any future accesses to virtual addresses between 4096 and 8191. Secondly, it would replace the cross in virtual page 8's entry with a 1, so that when the trapped instruction is re-executed, it will map virtual address 32780 to physical address 4108 ($4096 + 12$).

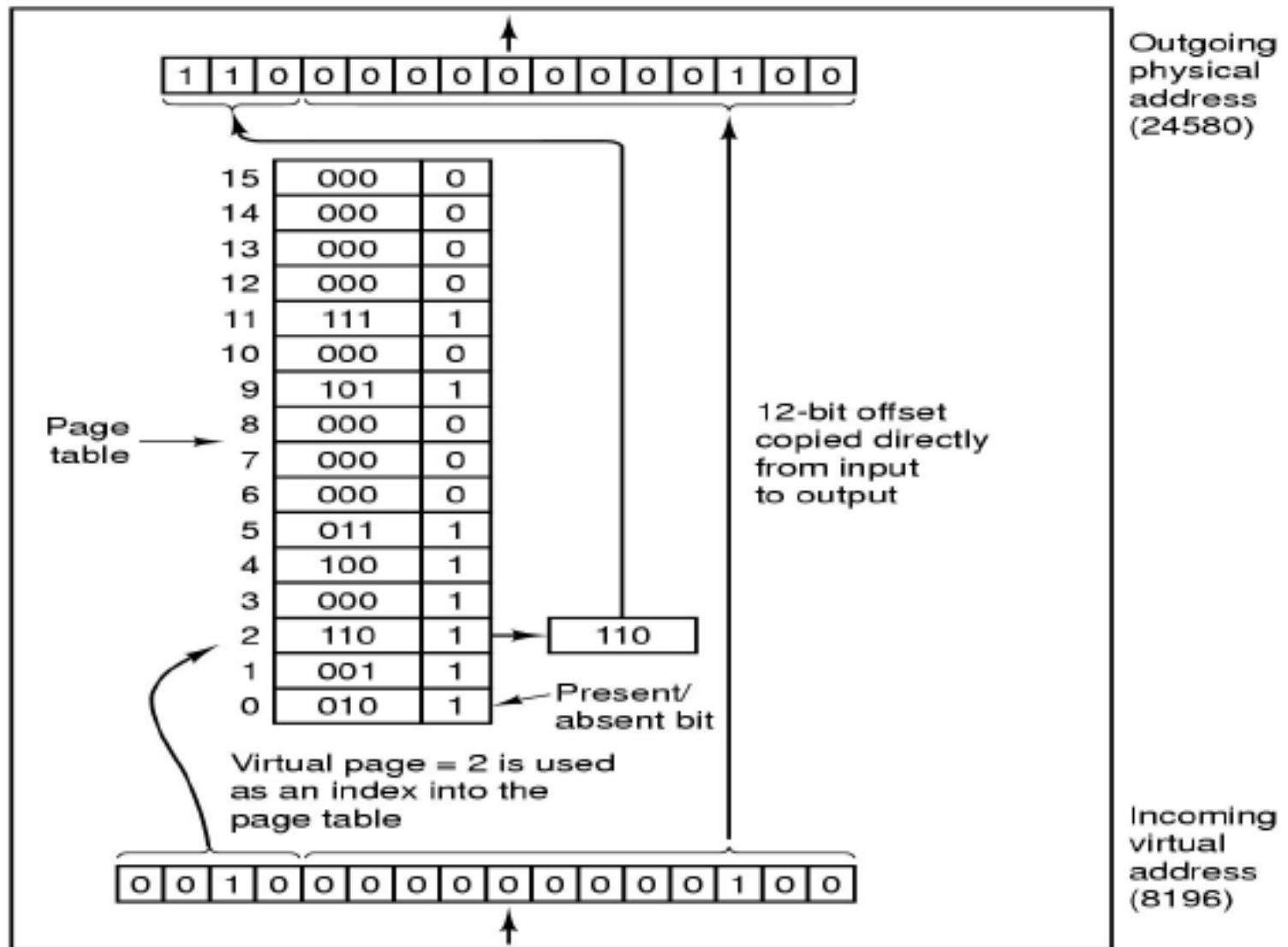
Address Translation – How MMU works

- Address generated by CPU (virtual address) is divided into two parts:
 - Page number (p): used as an index into a page table which contains base address of each page in physical memory.
 - Page offset (d): combined with base address to define the physical memory address that is sent to the memory unit.
- If the size of the logical address space is 2^m and page size is 2^n addressing units (bytes or words), then the high-order $(m-n)$ bits of logical address designate page number, and n lower-order bits designate the page offset.
- **Present/absent bit** keeps track of which pages are physically present in memory.

Address Translation



Address Translation Example



Explanation

- Here, we are supposing a computer that generates 16-bit addresses, from 0 up to 65535. These are the virtual addresses. So virtual address space is 2^{16} . Let the page sizes be of 4KB so that the page size is 2^{12} bytes. The page size and page frame have equal sizes.
- Now the page number will be the high-order 4 bits of the logical address and remaining 12 bits will be the page offset.
- Now let us see how the virtual address 8196 is translated to physical address.
- $8196 = 0010\ 000000000100$
- The page number is $0010 = 2$ i.e. the index 2 of the page table contains the base address of the physical memory where 8196 will be loaded, which is 110.
- Now combine the 12-bit from the offset to it to get the physical address = $110\ 000000000100$ which is 24580.
- **Note:** If the Present/Absent bit is 0, a trap to the operating system is caused. Only when the bit is 1, the above explanation holds.

Page Tables

- *For each process, page table stores the number of frame, allocated for each page.*
- The purpose of the page table is to map virtual pages into page frames.
- The function of page table can be represented in mathematical notation as:
 $page_frame = page_table(page_number)$
- The virtual page number is used as an index into the page table to find the corresponding page frame.

Structure of a Page Table Entry

Page Table Structure

The exact layout of page table entry is highly machine dependent, but more common structure for 32-bit system is as:

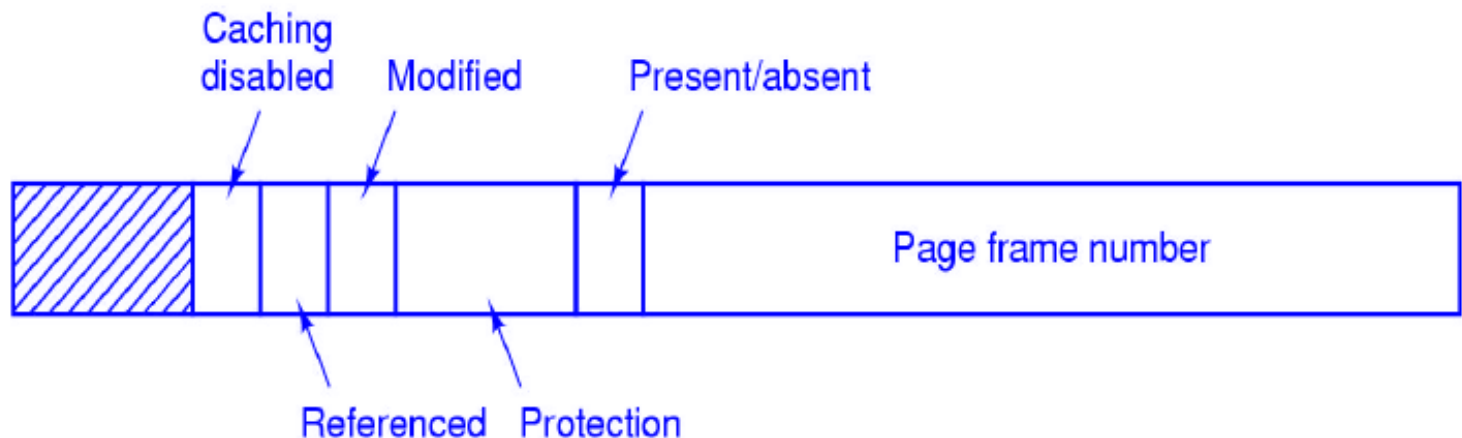


Fig: A typical page table entry

Structure of a Page Table Entry...

- The most important field is the *Page frame number*, since the goal of the page mapping is to output this value.
- **Present/absent bit:** If present/absent bit is 1, the virtual address is mapped to the corresponding physical address. If it is absent, a trap occurs, called page fault.
- **Protection bit:** Tells what kinds of access are permitted: 0 for read/write and 1 for read only.
- **Modified bit (dirty bit):** Identifies the changed status of the page since its last access. If the page have been written to or modified (i.e. is 'dirty'), the modified bit is set and the page must be written back to the disk, since at a later time the OS may reclaim this page frame. However, if it has not been modified (i.e. is 'clean'), the page frame can just be abandoned, since the disk copy is still valid.
- **Referenced bit:** It is set whenever a page is referenced. It is used in page replacement.
- **Caching disabled:** It allows caching to be disabled for the page. Used for that system where the mapping is onto device register rather than memory.

Page Table Issues

- **Size of page table:**

Most modern computers support a large virtual-address space (2^{32} to 2^{64}). If the page size is 4KB, a 32-bit address space has 1 million pages (2^{20}). With 1 million pages, the page table must have 1 million entries.

Think about 64-bit address space!!! A spontaneous idea that comes into mind is increase page size. But then there will be **large internal fragmentation**. For example, if pages are 16KB, a process of 72766 bytes would need 4 pages plus 7230 bytes. It would be allocated 5 frames, resulting in an internal fragmentation of $16384 - 7230 = 9154$ bytes. If the page sizes were 2KB, the resulting fragmentation would have been only 962 bytes.

- **Efficiency of mapping:**

If a particular instruction is being mapped, the table lookup time should be very small than its total mapping time to avoid becoming CPU idle.

What would be the performance, if such a large table have to load at every mapping.

How to handle these issues???

Multilevel Page Tables

- *To get around the problem of having to store huge page tables in memory all the time, many computers use the multilevel page table in which the page table itself is also paged.*
- Example: Pentium II - 2 level, 32-bit Motorola - 4 level, 32 -bit SPARC - 3 level, etc.
- Multilevel page tables more than 2 levels are generally considered inappropriate since they increase accessing complexity.

Example: Two-Level Page Tables

- Let we have a 32-bit virtual address space with a page size of 4KB.
- The virtual address space is partitioned into a 10-bit PT1 field, a 10-bit PT2 field, and a 12-bit offset field.
- There are a total of 2^{20} pages.



The top level page have 1024 entries, corresponding to PT1. At mapping, it first extracts the PT1 and uses this value as an index into the top level page table. Each of these entries have again 1024 entries, the resulting address of top-level yields the address or page frame number of second-level page table.

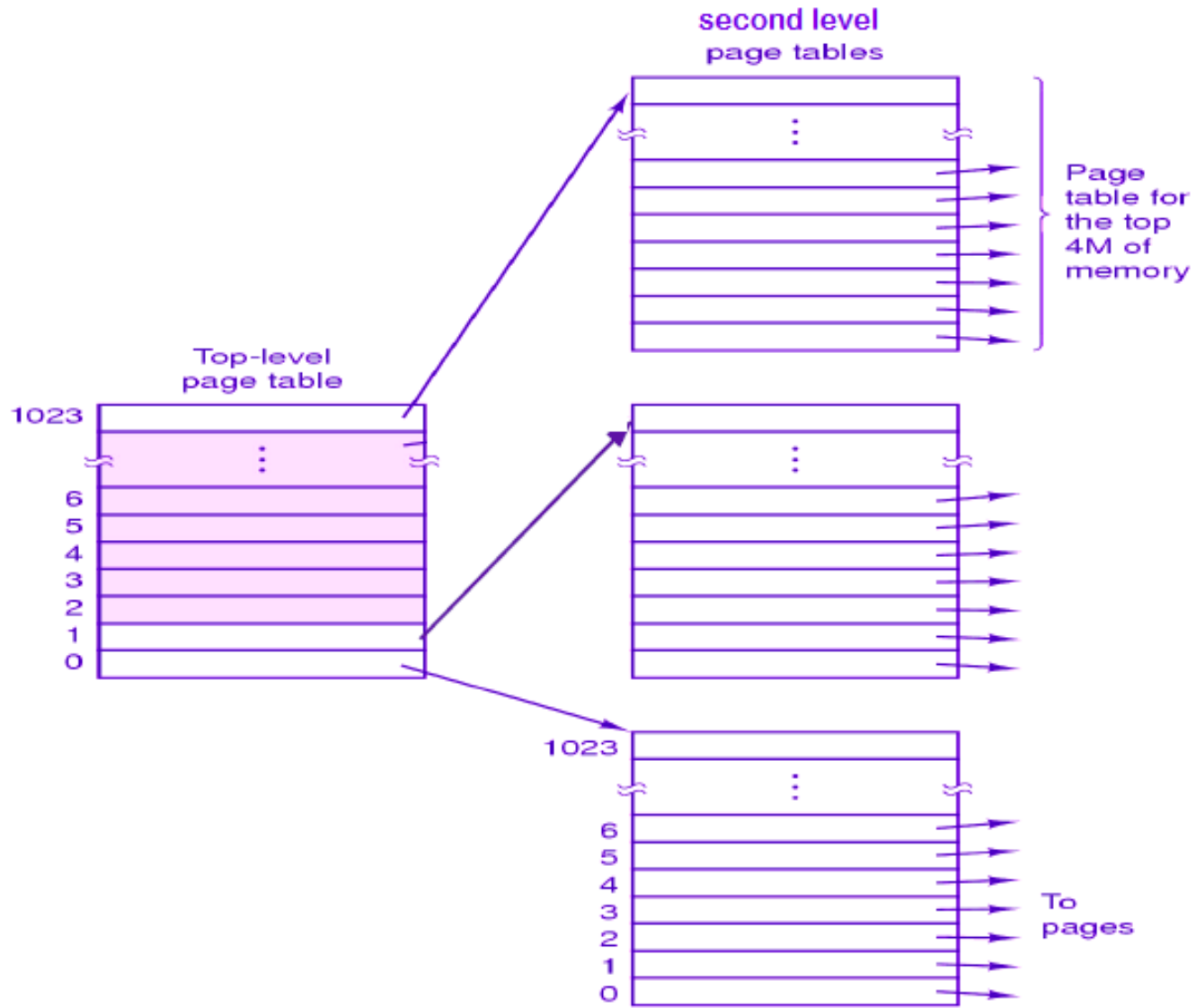


Fig: Two-level page tables

TU Exam Question 2068

A computer with 32-bit address uses a two level page table. Virtual addresses are split into a 8-bit top level page table field, 12-bit second level page table field and offset. How large the pages? How much maximum space is required when page tables loaded into memory if each entry required 4 bytes?

Answer: If first 8-bits of the virtual address serve as index into the first level page table and 12-bits are used as an index for second page table then 12-bits are used as an offset.

Now, top level page table contains 2^8 entries and second page table contains 2^{12} entries. Since one page table entry requires 4 byte, total space required to store both table into memory = $2^8 * 2^{12} * 4$ bytes = 4,194,304 bytes = 4096KB = 4 MB.

Hashed Page Tables

- A common approach for handling address space larger than 32-bits is to use a **hashed page table**, with the hash value being the virtual-page number.
- Each entry in the hash table contains a linked list of elements that hash to the same location (to handle collisions).
- Each element consists of three fields: virtual-page-number, value of mapped page frame, and a pointer to the next element.
- The algorithm works as follows: The virtual page number is hashed into the hash table. If there is match the corresponding page frame is used, and if not, subsequent entries in the linked list are searched for a matching virtual page number.

Hashed Page Tables

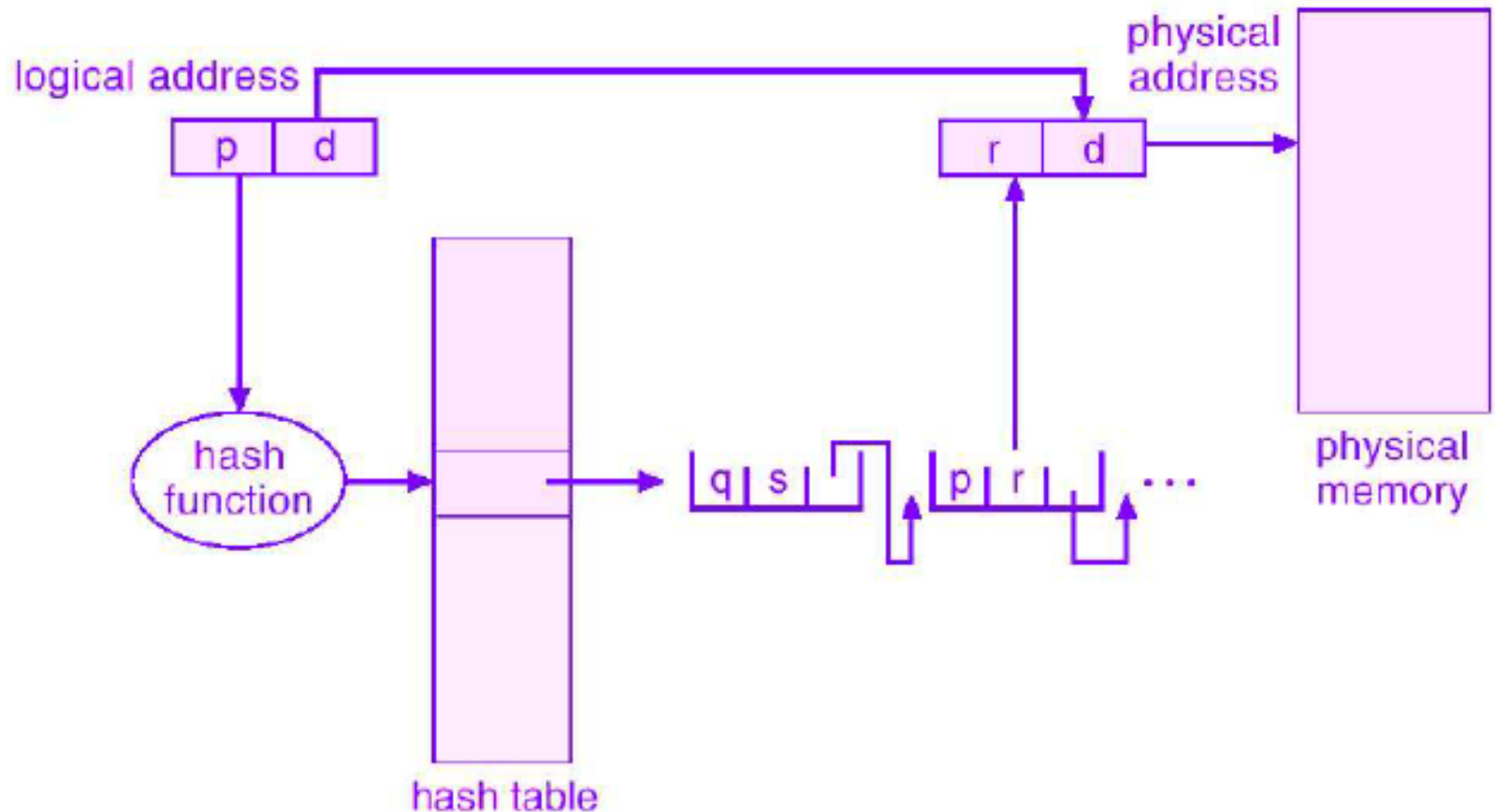
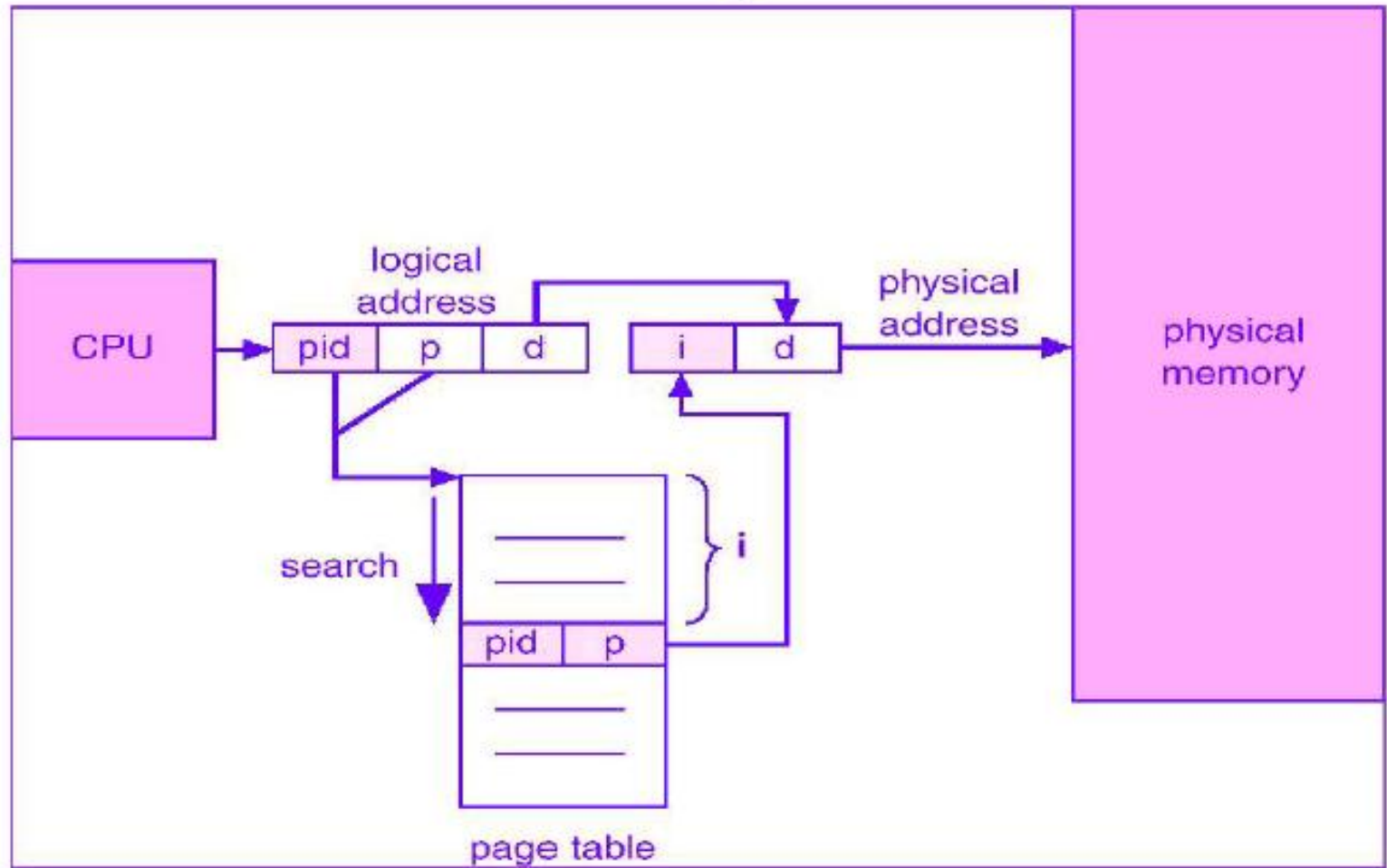


Fig: Hashed Page Table

Inverted Page Tables

- This is a common approach for handling address space larger than 32-bit.
- In this design, there is one entry per page frame in real memory, rather than one entry per page of virtual address space as in earlier tables. For example, with 64-bit virtual addresses, a 4KB page, and 1 GB of RAM, an inverted page table requires only 262,144 entries.
- Each entry consists of the virtual address of the page stored in that physical memory location, with information about the process that owns that page.
- Virtual address consists of three fields: [process-id, page-number, offset]. The inverted page table entry is determined by [process-id, page-number]. The page table is searched for the match, if say at entry i , the match is found, then the physical address [i , offset] is generated.

Inverted Page Tables



Inverted Page Tables...

- **Advantage:** Decreases the memory size to store the page table.
- **Problem:** It must search entire table in every memory reference, not just on page faults. *Searching a 256K table in every reference is not a way to make system fast!!!*
- **Solution:**
 - Use of Hashed tables.
 - Use of TLBs.

Translation Lookaside Buffers (TLBs)

- To speed up address translation, the following observation about locality is used: “**Most processes use a large number of references to a small number of pages and not the other way around**”.
- The small, fast, lookup hardware cache, called **TLB** is used to overcome this problem, by mapping logical address to physical address without page tables.
- The TLB is associative, high-speed, memory (usually included within MMU consisting of only 64 to 1024 entries); each entry consists information about virtual page-number and physical page frame number.
- Key of improvement: Parallel search for all entries.

Translation Lookaside Buffers (TLBs)

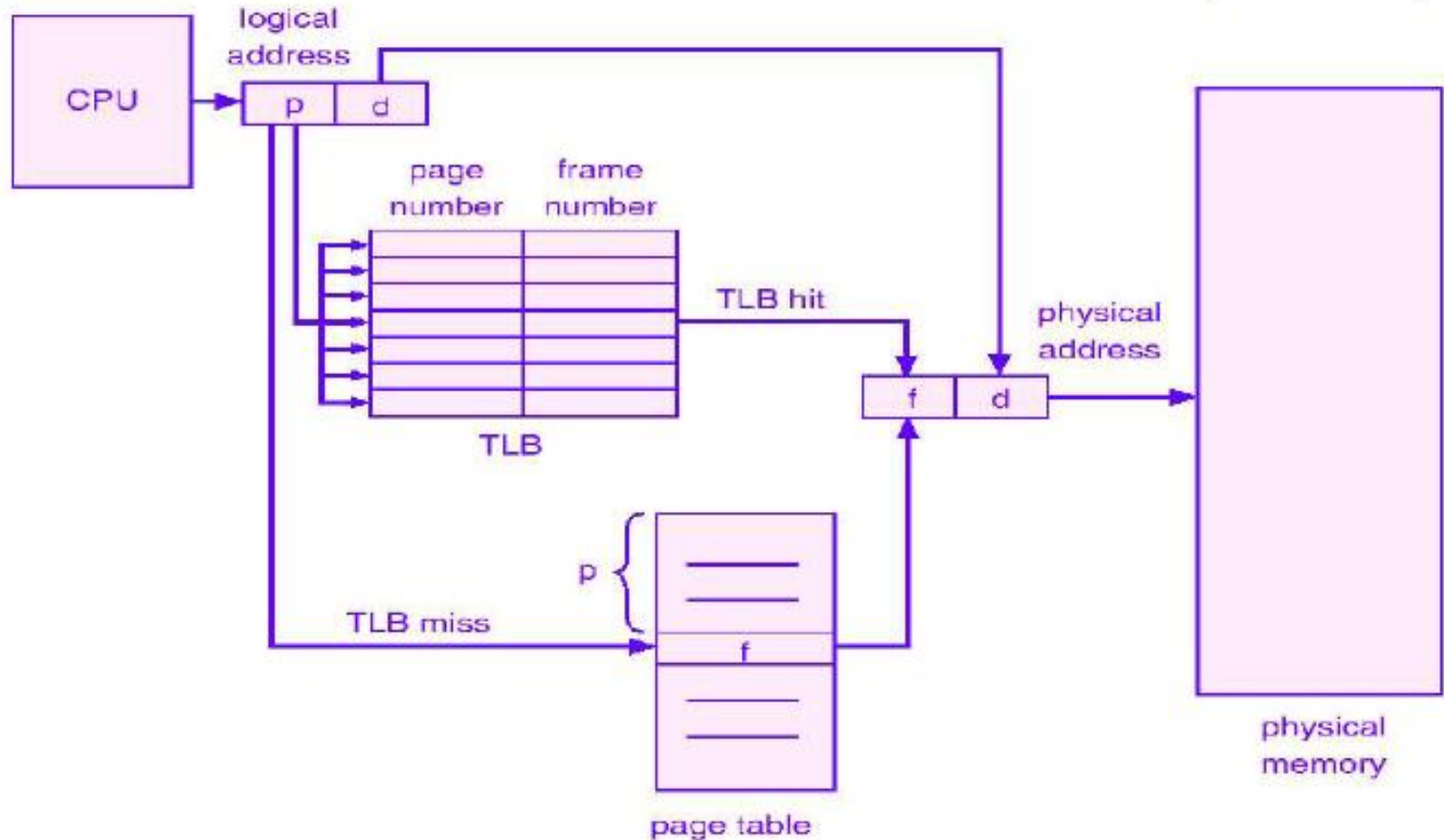


Fig: MMU with TLB

Translation Lookaside Buffers...

- When a virtual address is presented to the MMU for translation, the hardware first checks to see if its virtual page number is present in the TLB by comparing it to all the entries simultaneously (i.e. in parallel). If page number is found (**TLB hit**), its frame number is immediately available, the whole task would be very fast because it compares all TLB entries simultaneously.
- If the page number is not in TLB (**TLB miss**), a memory reference to the page table must be made. It then replaces one entry of TLB with the page table entry just referenced. Thus next time when the TLB is referenced for that page, **TLB hit** occurs.

Advantages/Disadvantages of Paging

- **Advantages:**

- Fast to allocate and free:
 - Allocate: Keep free list of free pages, grab first page in the list.
 - Free: Add pages to free list.
- Easy to swap-out memory to disk.
 - Frame size matches disk page size.
 - Swap-out only necessary pages.
 - Easy to swap-in back from disk.

- **Disadvantages:**

- Additional memory reference.
- Page tables are kept in memory.
- Internal fragmentation: process size does not match allocation size.

Assignment

1. Why are page sizes always a power of 2?
2. On a simple paging system with 2^{24} bytes of physical memory, 256 pages of logical address space, and a page size of 2^{10} bytes, how many bits are in a logical address?
3. Describe how TLB increase performance in paging.
4. Consider a logical address space of eight pages of 1024 words each, mapped onto a physical memory of 32 frames.
 - a. How many bits are there in the logical address?
 - b. How many bits are there in the physical address?

Answer 1: Paging is implemented by breaking up an address into a page and offset number. It is most efficient to break the address into X page bits and Y offset bits, rather than perform arithmetic on the address to calculate the page number and offset. Because each bit position represents a power of 2, splitting an address between bits results in a page size that is a power of 2.

Answer 2: 16

Answer 4: a. Logical address: 13 bits b. Physical address: 15 bits

Page Replacement

- When a page fault occurs, the OS has to choose a page to remove from memory to make the room for the page that has to be brought in.
- If the page to be removed has been modified while in memory, it must be rewritten to the disk to bring the disk copy up to date.
- If the page has not been changed, the disk copy is already up to date, so no rewrite is needed. The page to be read in just overwrites the page being removed.

TU Exam Question 2068/2067

- Under what circumstances do page fault occur? Describe the action taken by the operating system when a page fault occurs.
- **Answer:** [A page fault occurs when an access to a page is made that has not been brought into main memory. When a page fault occurs, a page frame is evicted from the main memory to make room for the requested page and then the page is loaded into the page frame.]

Page Replacement Algorithms

- The page replacement algorithms deal with:
 - Which one page has to be removed?
 - What happens if the page that required next, is removed?
 - How to decrease the page fault rate?
- Principle of Optimality – *“To obtain optimal performance, the page to replace is the one that will not be used for the farthest time in future.”*

The Optimal Page Replacement Algorithm

- **Key Idea:** Replace the page that will not be used for the longest period of time.
- Each page can be labeled with the number of instructions that will be executed before that page is first referenced.
- For example, for 3 page frames and 8 pages system, the optimal page replacement would yield nine page faults on our sample reference string as:

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2		2		2						7		
	0	0	0		0		0		0		0						0		
		1	1		3		3		3		1						1		

page frames

The Optimal Page Replacement Algorithm...

- The first three reference causes faults that fill the three empty frames.
- The reference to page 2 replaces page 7, because 7 will not be used until reference 18, whereas page 0 will be used at 5, and page 1 at 14.
- Similarly, the reference to page 3 replaces page 1, as page 1 will be the last of the three pages in memory to be referenced again and so on.
- **Advantage:** It guarantees the lowest possible page fault rate.
- **Disadvantage:** Unrealizable because at the time of the page fault, the OS has no way of knowing when each of the pages will be referenced next. So this is not used in practical system, however it can be used to compare the performance of other page replacement algorithms.

The First-In, First-Out (FIFO) Page Replacement Algorithm

- Associate each page with the time, when that page was brought into the memory. The page with the highest time is chosen to replace.
- This can be implemented by using queue of all pages in memory.
- Example:

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2
	0	0	0
		1	1

2	2	4	4	4	0
3	3	3	2	2	2
1	0	0	0	3	3

0	0
1	1
3	2

7	7	7
1	0	0
2	2	1

page frames

Note: While tracing, provide time to each pages when they come (starting from 20 i.e. equal to length of reference string in this example)

The First-In, First-Out (FIFO) Page Replacement Algorithm...

- **Advantages:**

- Easy to understand and program.

- Distributes fair chance to all.

- **Problems:**

- FIFO is likely to replace heavily (or constantly) used pages which are still needed for further processing.
 - Increased page fault rate (in above case 15).

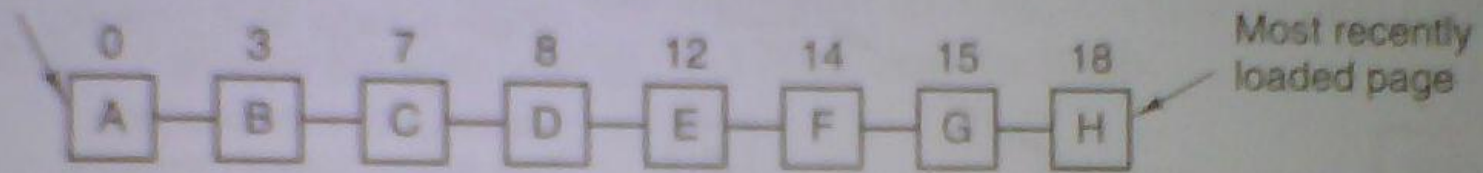
TU Model Question

- Write a short note on *The First-In, First-Out (FIFO) Page Replacement Algorithm*.

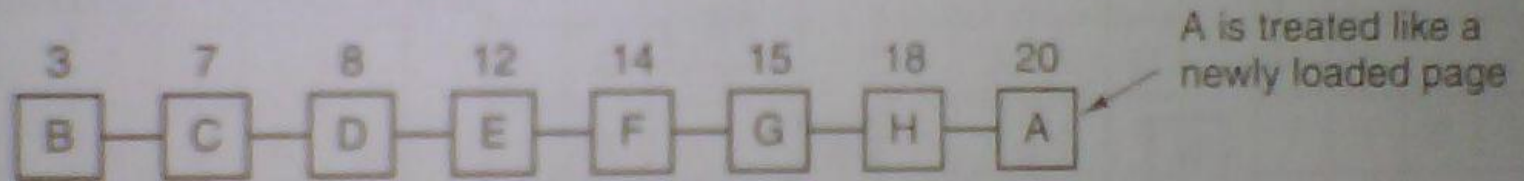
The Second-Chance Page Replacement Algorithm

- A simple modification to FIFO that avoids the problem of throwing out a heavily used page is to inspect the referenced bit (R) of the oldest page.
- If the R bit is 0, the page is both old and unused, so it is replaced immediately.
- If the R bit is 1, the page is given a **second chance**. When a page gets a second chance, its R bit is cleared and its arrival time is reset to the current time so that the page is put at the end of the list of pages. Thus, a page that is given a second chance will not be replaced until all other pages are replaced (or given second chances).

Page loaded first



(a)



(b)

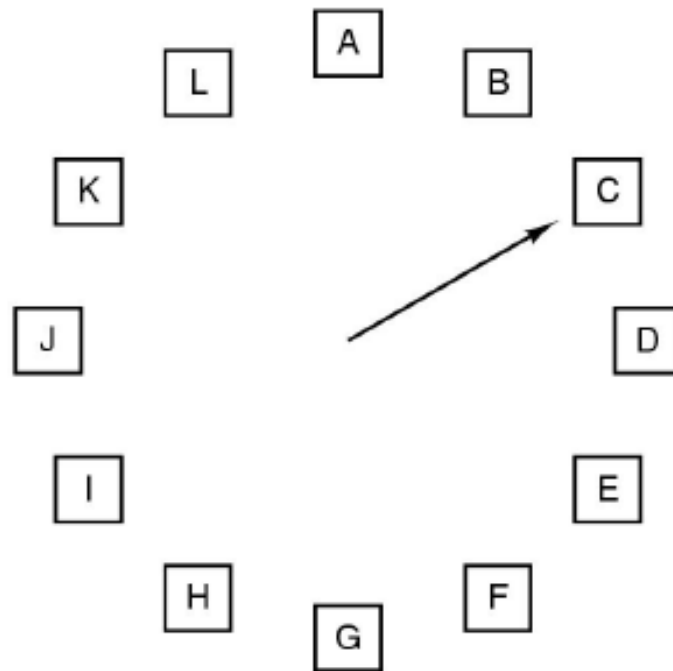
Figure _____ Operation of second chance. (a) Pages sorted in FIFO order. (b) Page list if a page fault occurs at time 20 and A has its *R* bit set. The numbers above the pages are their load times.

Advantage: Big improvement over FIFO.

Problem: If all the pages have been referenced, second chance degenerates into pure FIFO.

The Clock Page Replacement Algorithm

- Although second chance is a reasonable algorithm, it is unnecessarily inefficient because it is constantly moving pages around the list.
- Better approach: Arrange the page frames in a circular list in the form of a **clock** instead of a linear list. The hand points to the oldest page.
- Differs from second chance only in implementation.
- **Advantage:** More efficient than second chance.



When a page fault occurs,
the page the hand is
pointing to is inspected.
The action taken depends
on the R bit:
R = 0: Evict the page
R = 1: Clear R and advance hand

Fig: Clock Page Replacement Algorithm

The Least Recently Used (LRU) Page Replacement Algorithm

- Recent past is a good indicator of the near future: pages that have been heavily used in the last few instructions will probably be heavily used again in the next few, while pages that have not been used for ages will probably remain unused for a long time.
- Hence, when a page fault occurs, throw out the page that has been unused for longest time.
- It maintains a linked list of all pages in memory with the most recently used page at the front and least recently used page at the rear. The list must be updated on every memory reference.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

Advantage: Excellent; efficiency is close to the optimal algorithm.

Problems:

Difficult to implement.

If it is implemented as a linked list of all pages in memory, updating list in every memory reference is not a way of making system fast!!!

Alternate implementation is by special hardware. For this, it requires a ***time-of-use*** field in page table and a logical clock or counter in the CPU.

Another Hardware LRU implementation: For a machine with n page frames, the LRU hardware can maintain a matrix of $n \times n$ bits, initially all zero. Whenever page frame k is referenced, the hardware first sets all the bits of row k to 1, then sets all the bits of column k to 0. At any instant, the row whose binary value is lowest is the least recently used, the row whose value is next lowest is next least recently used and so on.

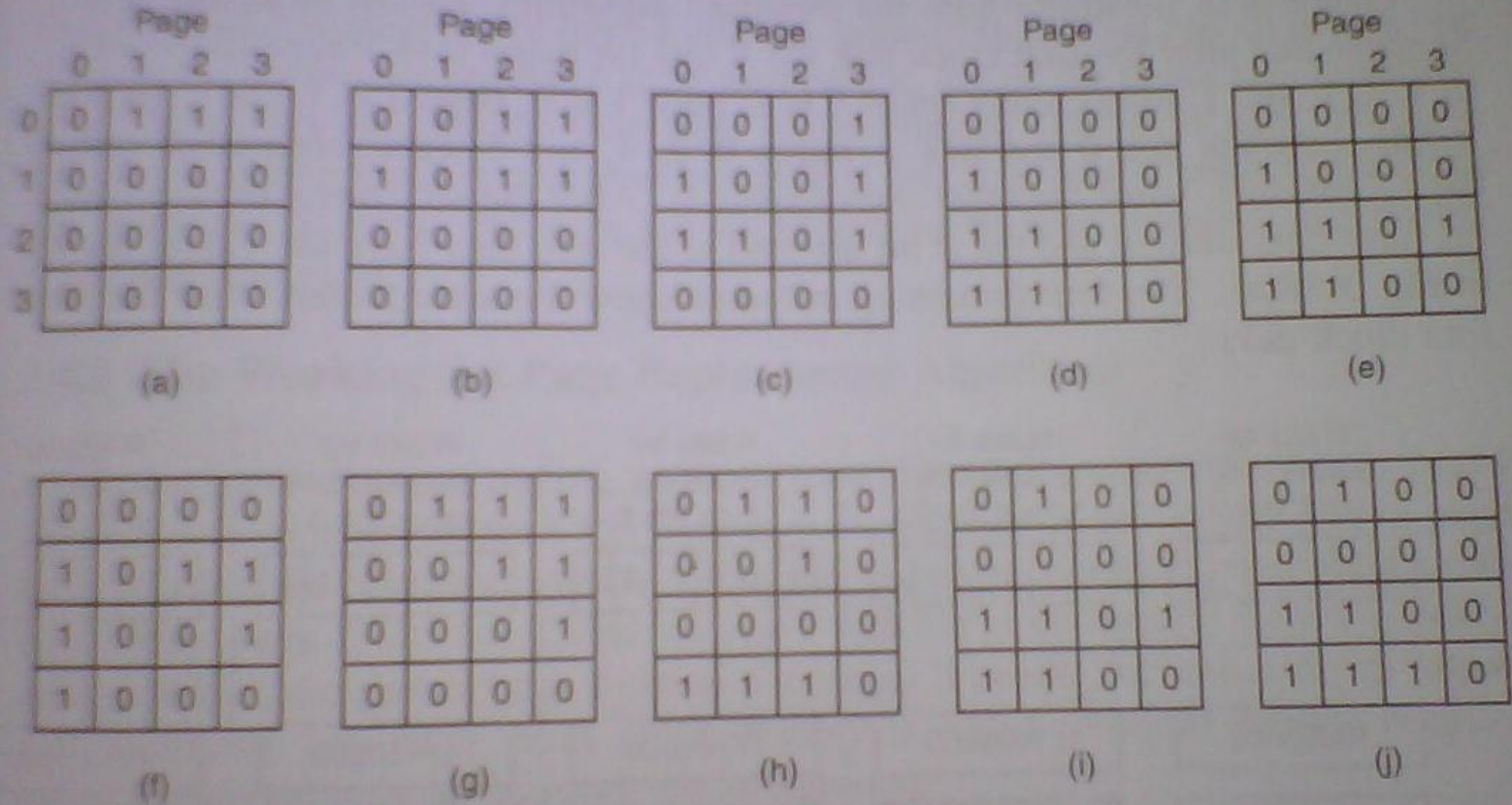


Figure LRU using a matrix when pages are referenced in the order 0, 1, 2, 3, 2, 1, 0, 3, 2, 3.

The Least Frequently Used (LFU) Page Replacement Algorithm

- One approximation to LRU's software implementation. It requires a software counter associated with each page.
- LFU requires that the page with the smallest count be replaced. The reason for this selection is that an actively used page should have a large reference count.
- When page fault occurs, the page with the lowest count is chosen for replacement.
- **Problem:** This algorithm suffers from the situation in which a page is used heavily during the initial phase of a process, but then is never used again. Since it was used heavily, it has a large count and remains in memory even though it is no longer needed.

The Not Recently Used Page Replacement Algorithm

- LRU approximation by enhancing second chance.
- *Pages not recently used are not likely to be used in near future and they must be replaced with incoming pages.*
- To keep useful statistics about which pages are being used and which pages are not, most computers have two status bits associated with each page - **referenced** and **modified**.
- These bits must be updated on every memory reference.
- When a page fault occurs, the OS inspects all the pages and divides them into four categories based on the current values of their referenced and modified bits:
 - Class 0: not referenced, not modified.
 - Class 1: not referenced, modified.
 - Class 2: referenced, not modified.
 - Class 3: referenced, modified.
- Pages in the lowest numbered class should be replaced first, and those in the highest numbered groups should be replaced last.
- Pages within the same class are randomly selected.
- **Advantage:** Easy to understand and efficient to implement.
- **Problem:** Class 1 is unrealistic.

The Working Set Page Replacement Algorithm

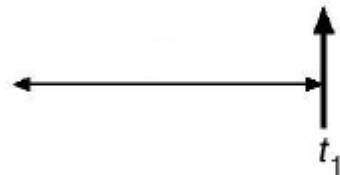
- In multiprogramming, processes are frequently moved to disk (i.e. all their pages are removed from memory) to let other processes have a turn at the CPU.
- What to do when a process has just been swapped out and another process has to load? A simple thing to do is to allow page faults until the required pages for the process are loaded (called **demand paging**) but allowing page faults is slow and it wastes considerable CPU time. Another thing that can be done is with the concept of working set of a process.
- The set of pages that a process is currently using is called its **working set**.
- If the entire working set is loaded in memory before letting the process run, then the process will run without causing many faults. Otherwise, excessive page faults will occur called **thrashing**.
- Many paging systems try to keep track of each process's working set and make sure that it is in memory before the process runs - **working set model** or **prepaging**.

The Working Set Page Replacement Algorithm

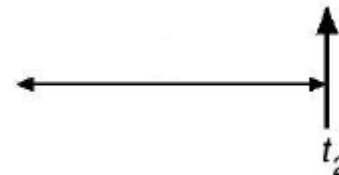
- The working set of pages of a process, $WS(t)$ at time t , is the set of pages referenced by the process in time interval $(t-k)$ to t .
- The variable k is called the **working-set-window**, and the size of k is the central issue in this model.
- Ex: Working-set-model with $k = 10$.

page reference table

... 2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



$WS(t_1) = \{1, 2, 5, 6, 7\}$



$WS(t_2) = \{3, 4\}$

The Working Set Page Replacement Algorithm

- To implement working set model, it is necessary for the OS to keep track of which pages are in the working set. This leads to a possible page replacement algorithm: when a page fault occurs, find a page in the working set and evict it.
- One efficient approximation technique used for implementing the working set page replacement algorithm is to define the working set as the set of those pages that were referenced during the past τ seconds of virtual time.
- The amount of CPU time a process has actually used since it started is called its **current virtual time**.

Working Set (WS) Page Replacement

2204 Current virtual time

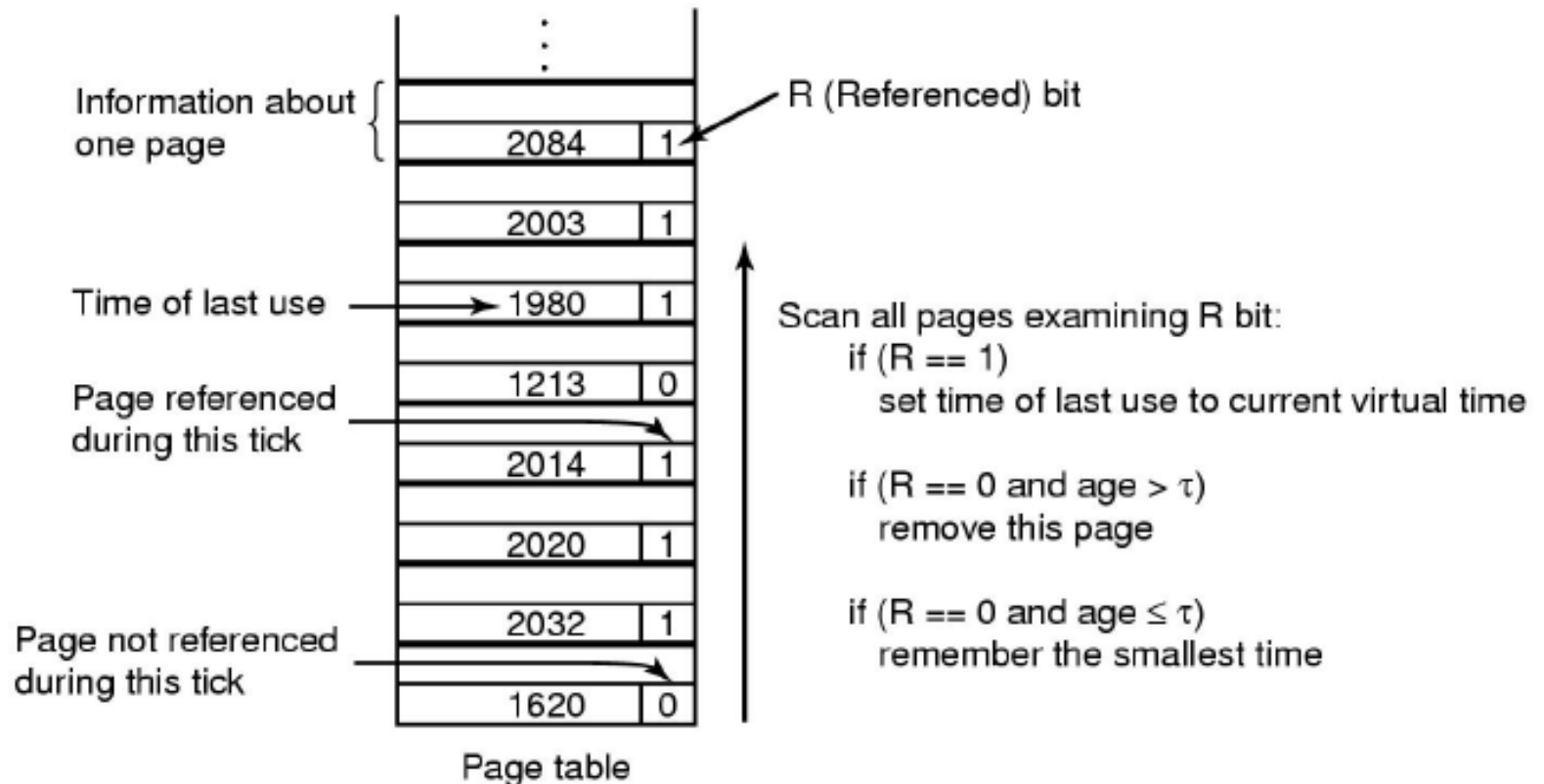


Fig: Page replacement with working set model

Algorithm Explanation

- Each entry of the page table contains two key items of information: the time the page was last used and the R bit. The empty rectangle symbolizes other fields not needed for this algorithm like page frame number, modified bit, etc.
- The R bit is cleared at every clock interrupt.
- On every page fault, the page table is scanned to look for a suitable page to evict.
- As each entry is processed, the R bit is examined. If it is 1, the current virtual time is written into the ***Time of last use*** field in the page table.
- If R is zero, the page is not referenced during the current clock tick so that it may be a candidate for removal. Its age (the ***current virtual time*** minus its ***Time of last use***) is computed and compared to τ . If ***age*** $> \tau$, the page is no longer in the working set so that new page replaces it. The scan continues updating the remaining entries.
- If R is zero and ***age*** $\leq \tau$, the page is still in the working set. The page with the greatest age (smallest ***Time of last use*** field) is noted.
- If all pages are in the working set, then find entries with R=0 and remove the page with greatest age.
- If all pages have been referenced during the current clock tick (i.e. all have R=1), one is chosen at random for removal, preferably a clean page.

The WSClock Page Replacement Algorithm

- The basic working set algorithm is cumbersome, since the entire page table has to be scanned at each page fault until a suitable candidate is located.
- An improved algorithm, that is based on the clock algorithm but also uses the working set information, is called WSClock. It is efficient and widely used algorithm.
- The data structure needed is a circular list of page frames. Initially, the list is empty. When the first page is loaded, it is added to the list. As more pages are added, they go into the list to form a ring. Each entry contains the ***Time of last use*** field and the **R** bit.
- At each page fault, the page pointed to by the hand is examined first. If $R=1$, the page has been used during the current tick, so clear R bit and advance hand to next page.
- If $R=0$ and $\text{age} > \tau$, put new page there.
- Other conditions similar to previous (WS Page Replacement).

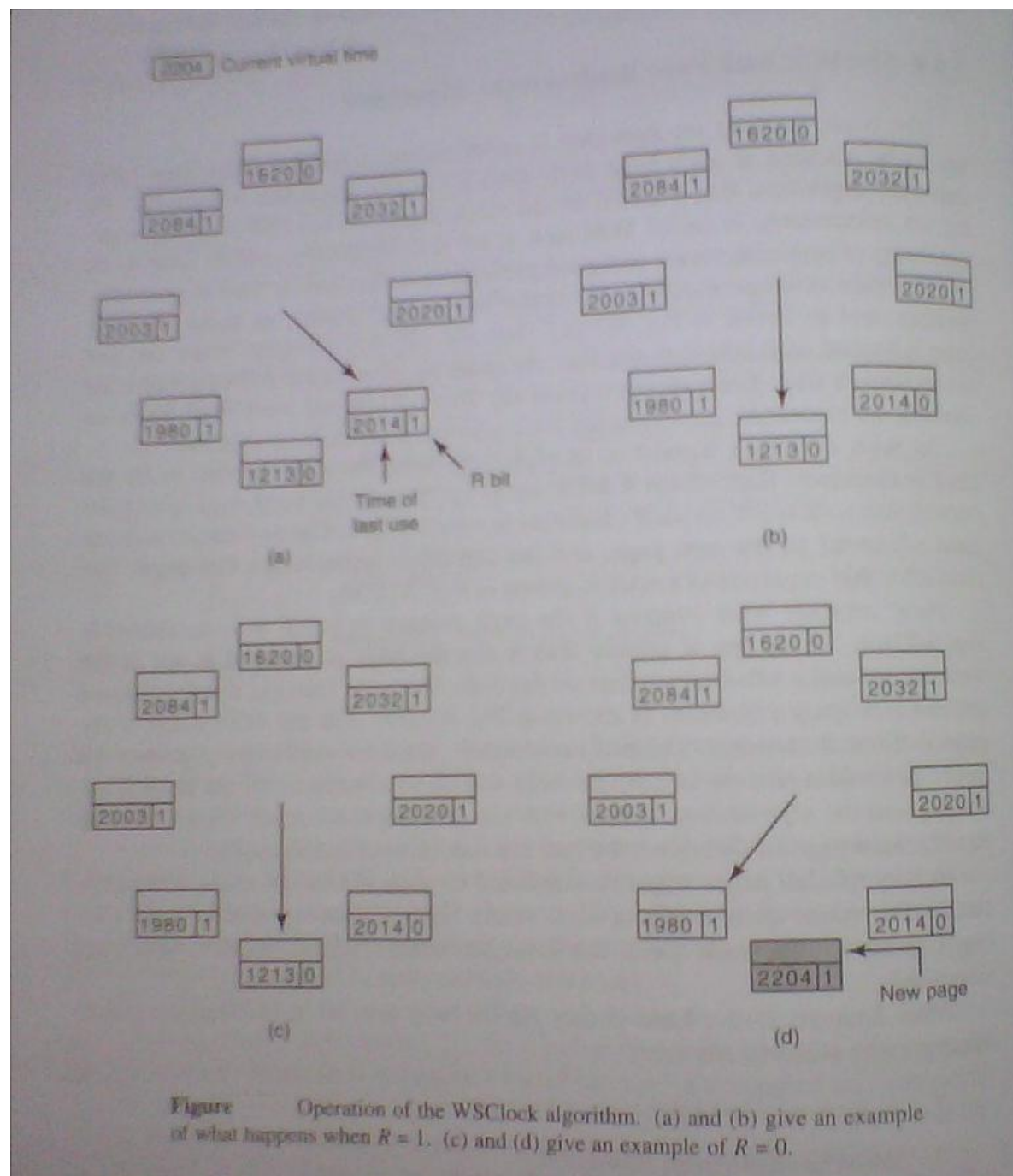


Figure Operation of the WSClock algorithm. (a) and (b) give an example of what happens when $R = 1$. (c) and (d) give an example of $R = 0$.

TU Exam Question 2067

Consider the following page reference string : 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6. How many page faults would occur for the LRU replacement, FIFO replacement and optimal replacement algorithms assuming one, two, **three**, four, **five**, six or **seven** frames? Remember all frames are initially empty, so your first unique pages will all cost one fault each.

Answer:

<u>Number of frames</u>	<u>LRU</u>	<u>FIFO</u>	<u>Optimal</u>
1	20	20	20
2	18	18	15
3	15	16	11
4	10	14	8
5	8	10	7
6	7	10	7
7	7	7	7

LRU replacement for three frames:

1	2	3	4	2	1	5
1	1	1	4	X	4	5
	2	2	2	X	2	2
		3	3	X	1	1

6	2	1	2	3	7	6
5	5	1	X	1	7	7
6	6	6	X	3	3	3
1	2	2	X	2	2	6

3	2	1	2	3	6
X	2	2	X	X	2
X	3	3	X	X	3
X	6	1	X	X	6

FIFO replacement for three frames:

1	2	3	4	2	1	5
1	1	1	4	X	4	4
	2	2	2	X	1	1
		3	3	X	3	5

6	2	1	2	3	7	6
6	6	6	X	3	3	3
1	2	2	X	2	7	7
5	5	1	X	1	1	6

3	2	1	2	3	6
X	2	2	X	2	6
X	7	1	X	1	1
X	6	6	X	3	3

Optimal replacement for three frames:

1	2	3	4	2	1	5
1	1	1	1	X	X	1
	2	2	2	X	X	2
		3	4	X	X	5

6	2	1	2	3	7	6
1	X	X	X	3	3	X
2	X	X	X	2	7	X
6	X	X	X	6	6	X

3	2	1	2	3	6
X	3	3	X	X	3
X	2	2	X	X	2
X	6	1	X	X	6

Segmentation

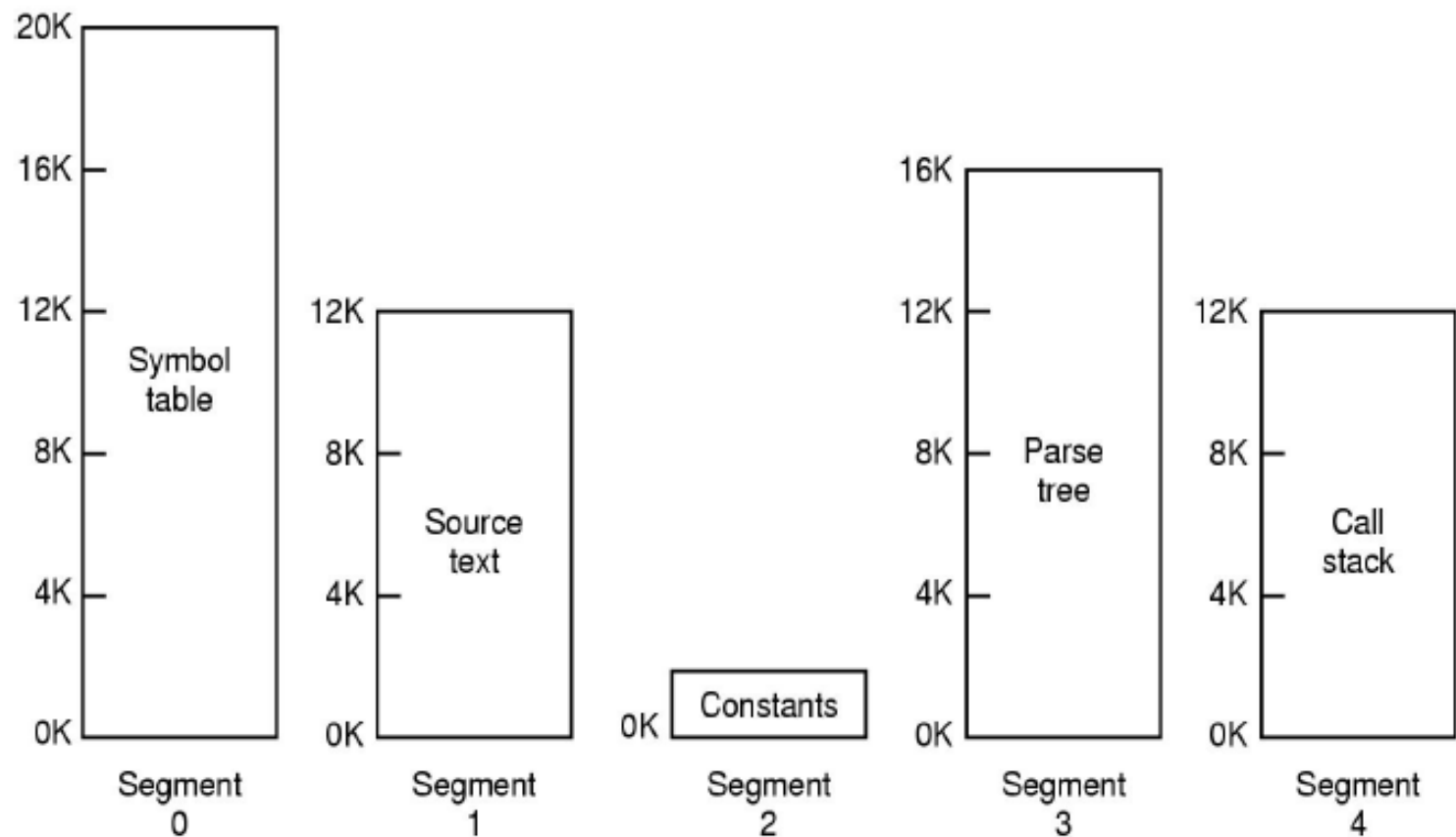
- **Issues:**

- What happens if programs increase their size in their execution?
- How to manage expanding and contracting tables?
- How to protect only data from the program?
- How to share data to other program or functions?
- The general solution of these issues is to provide the machine with many completely independent address spaces (rather than a one-dimensional virtual address space), called **segments**.

Segmentation...

- Segmentation is a memory management scheme that supports variable partitioning and mechanisms with freedom of contiguous memory requirement restriction.
- The independent block of a program is a **segment** such as: main program, procedures, objects, local variables, global variables, stacks, symbol tables, arrays, etc.
- The responsibility for dividing the program into segments lies with user (or compiler).

Segmentation



Segmentation...

- Different segments have their own name and size.
- The different segments can grow or shrink independently, without affecting the others; so the size of segment changes during execution.
- For the simplicity of implementation, segments are numbered and are referred to by a segment number, rather than by segment name. Thus the logical address consists of a **segment number** and an **offset**.
- The segment table (like page table but each entry consists of **limit** and **base register** value) is used to map the logical address to physical address.

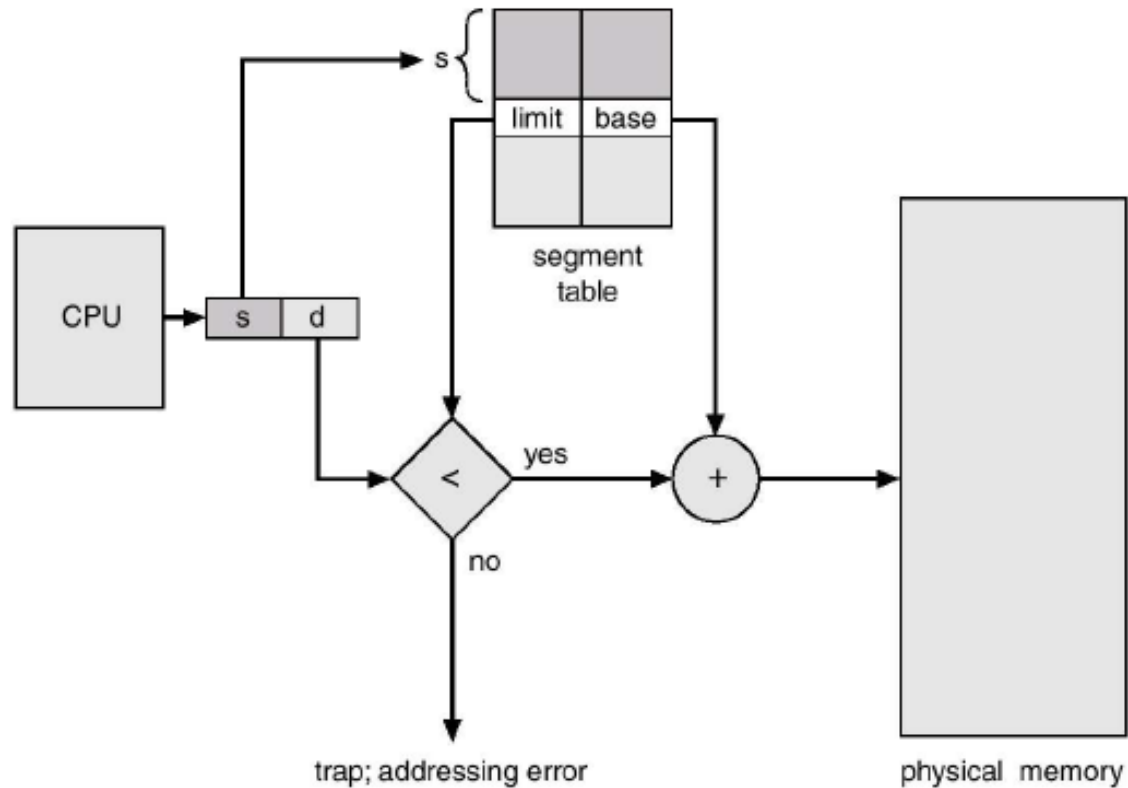


Fig: Segmentation

*The segment number is used as an index into the segment table. The offset **d** of the logical address must be between 0 and the segment limit. If it is not, trap occurs, if it is legal, it is added to the segment base to produce the address in the physical memory.*

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

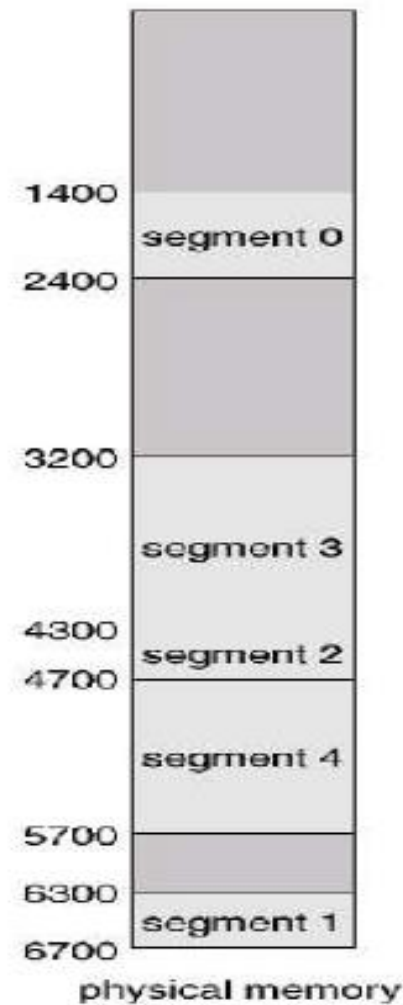


Fig: Example of Segmentation

Note: The segmentation scheme causes fragmentation, this can be handled by the same technique of variable partition memory management.

Class work

- Consider the following segment table:

<u>Segment</u>	<u>Base</u>	<u>Size</u>
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?

- a) 0430
- b) 110
- c) 2500
- d) 3400
- e) 4112

Answer:

- a. $219 + 430 = 649$ (0 is segment number, 430 is offset and $649 < 819$)
- b. $2300 + 10 = 2310$
- c. illegal reference, trap to operating system
- d. $1327 + 400 = 1727$
- e. illegal reference, trap to operating system

TU Exam Question 2066

- Explain the mapping of virtual address to real address under segmentation.

Comparison of paging and segmentation

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

More about segmentation...

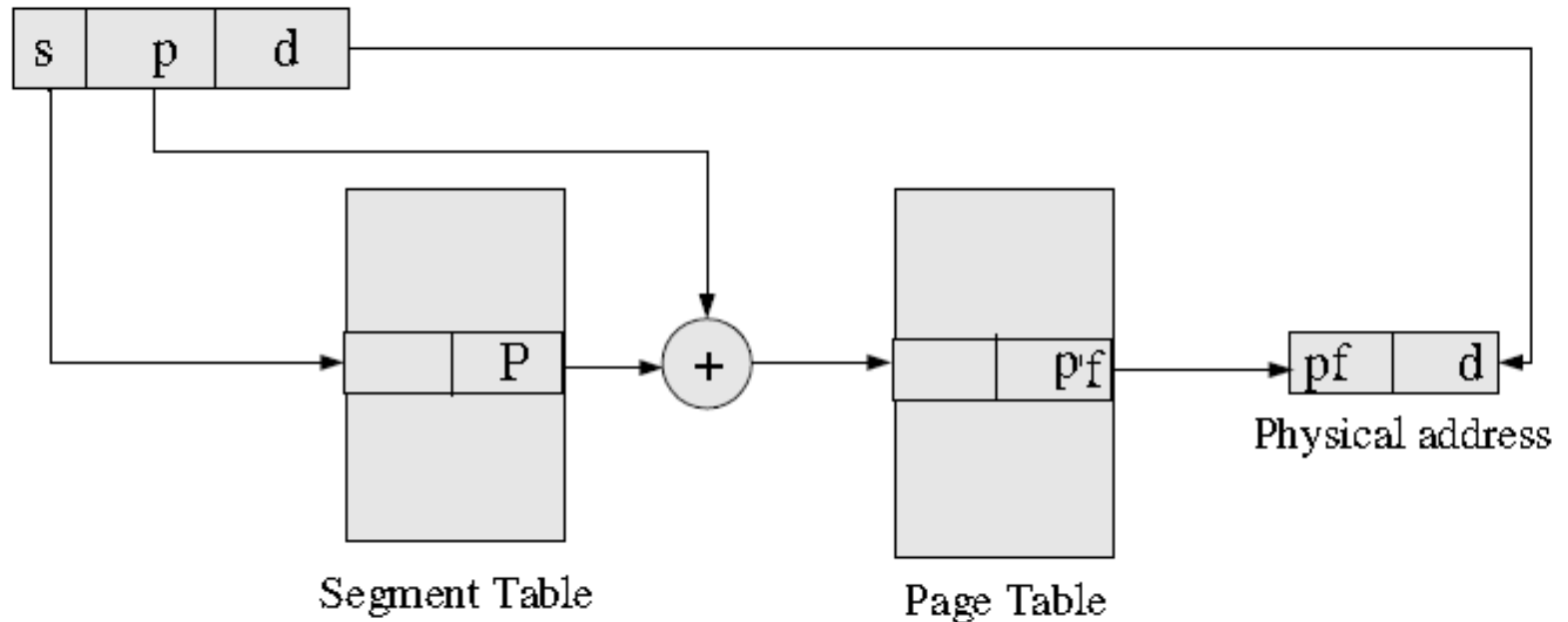
- ***How one segment could belong to the address space of two different processes?***
- Since segment tables are a collection of base–limit registers, segments can be shared when entries in the segment table of two different processes point to the same physical location. The two segment tables must have identical base pointers, and the shared segment number must be the same in the two processes.

Segmentation with Paging

- What happens when segments are larger than main memory? – Segmentation with Paging.
- Segmentation can be combined with paging to provide the efficiency of paging with the protection and sharing capabilities of segmentation.
- As with simple segmentation, the logical address specifies the segment number and the offset within the segment.
- When paging is added, the segment offset is further divided into a page number and page offset.
- The segment table entry contains the address of the segment's page table.

Segmentation with Paging

Logical address



How segmentation with paging works?

- Every process is divided into many segments.
- The segment table is maintained to keep track of the segments for a process. Every segment is provided a segment ID.
- Each of the segment have their corresponding page table. If a segment's page table is not in memory, a segment fault occurs. If it is, then the page table for the segment is located.
- The page table entry for the requested virtual page is then checked. If the page is not in memory, a page fault occurs. If it is, the physical address of the start of page is extracted from the page table entry.
- Finally, actual physical address is the address, start of page plus offset.
- Now read or write operation can be performed.

Segmentation with Paging...

Examples:

- ***The Intel Pentium:*** The Intel Pentium 80386 and later architecture uses segmentation with paging memory management. The maximum number of segments per process is 16K, and each segment can be as large as 4 GB. The page size is 4KB. It uses two-level paging scheme.
- ***MULTICS:*** It has 256K independent segments, and each up to 64KB. The page size is 1KB or small.

TU Exam Question 2068

- What do you mean by memory fragmentation? Distinguish between the internal and external fragmentation.
- ***Hint:*** *[Internal Fragmentation is the area in a region or a page that is not used by the process occupying that region or page. This space is unavailable for use by the system until that process is finished and the page or region is released.]*

If a process requests memory and if it does not get exactly the memory as it needs to complete its execution, although there exists enough non-contiguous memory to satisfy the request is called external fragmentation. This problem can be solved by the memory compaction or coalescing.]

TU Exam Question 2066

- Write short notes on :
 - a. Least recently used page replacement algorithm
 - b. Segmentation
 - c. Associative memory